

Universidad Europea Miguel de Cervantes

Escuela Politécnica Superior

Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

Detección de Phishing en Correos Electrónicos

Autor: Alejandro Villarrubia García

Tutor: Carmelo González García

2026

Resumen

El phishing se ha consolidado como una de las ciberamenazas más persistentes y dañinas. Este Trabajo Fin de Grado diseña, implementa y evalúa un sistema cliente-servidor para detectar phishing en correo. Streamlit, la extensión y el monitor envían las entradas a una API central que combina heurística explicable y modelos TF-IDF + MLP en español e inglés. El servidor es el único responsable de parsear, analizar, entrenar y versionar los modelos, de modo que una actualización se aplica a todos los clientes. La memoria revisa el estado del arte y presenta una validación reproducible, delimitando expresamente sus límites frente a producción.

Palabras clave: phishing, ciberseguridad, ingeniería social, machine learning, deep learning, detección de correo electrónico.

Abstract

Phishing remains one of the most persistent and damaging cybersecurity threats. This Bachelor's Thesis designs, implements and evaluates a client-server system for phishing email detection. The browser uses Streamlit as the presentation layer, while Streamlit, the Gmail extension and the monitor send text, EML or structured fields through HTTP to a mandatory central backend. Only the server parses messages, combines explainable heuristics with Spanish and English TF-IDF + MLP models, trains them and keeps one active version per language, so an update is shared by every client. The prototype is validated with automated tests, real browser-to-backend journeys, separate calibration and a reserved local EML scenario corpus, while its limits for production are stated explicitly.

Keywords: phishing, cybersecurity, social engineering, machine learning, deep learning, email detection.

Agradecimientos

Quiero expresar mi agradecimiento a mi tutor, Carmelo González García, por su orientación, sus comentarios y su apoyo constante durante la realización de este trabajo. Agradezco también a mi familia y amigos su comprensión y ánimo a lo largo de toda la carrera, así como a todas las personas que de una u otra forma han contribuido a que este proyecto fuese posible.

Índice de figuras y tablas

Figuras

Figura 2.1: Diagrama de flujo swimlane de las fases de un ataque de phishing.

Figura 2.2: Vectores iniciales en brechas analizadas por el DBIR 2025 (n=9.891).

Figura 5.1: Arquitectura general cliente-servidor del prototipo.

Figura 5.2: Flujo de análisis desde la entrada hasta la respuesta del backend.

Figura 6.1: Pantalla inicial del cliente web con el backend central conectado.

Figura 6.2: Fuentes de entrada y configuración del modo combinado en el cliente web.

Figura 6.3: Resultado combinado del escenario BEC sintético reservado.

Figura 6.4: Resultado heurístico y desglose explicable de señales por categoría.

Figura 6.5: Administración y versiones activas de los modelos centrales ES y EN.

Figura 6.6: Ajustes de análisis e hiperparámetros almacenados en el servidor.

Tablas

Tabla 2.1: Indicadores de contexto del phishing y las brechas (2025).

Tabla 2.2: Comparativa de modelos.

Tabla 2.3: Controles por fase.

Tabla 2.4: Aplicación de los principios de Cialdini en campañas de phishing.

Tabla 3.1: Comparativa de técnicas clásicas de machine learning.

Tabla 3.2: Comparativa de técnicas de deep learning y transformers.

Tabla 4.1: Planificación temporal del proyecto.

Tabla 4.2: Herramientas y tecnologías utilizadas.

Tabla 5.1: Cambios respecto a la propuesta inicial.

Tabla 5.2: Capas y responsabilidades del diseño modular.

Tabla 6.1: Conjuntos empleados y separación experimental.

Tabla 6.2: Métricas sobre los holdouts de texto.

Tabla 6.3: Métricas sobre los 16 EML reservados.

Tabla 6.4: Matrices de confusión sobre los 16 EML reservados.

Abreviaturas

API: Interfaz de operaciones que permite a un cliente enviar datos al backend mediante un contrato HTTP; Gmail y Telegram son integraciones externas distintas.

AiTM: Adversary-in-the-Middle; ataque que se sitúa entre el usuario y el servicio legítimo para interceptar la autenticación.

BEC: Business Email Compromise; fraude que suplanta a directivos o proveedores.

BERT: Bidirectional Encoder Representations from Transformers; modelo de lenguaje pre-entrenado.

CNN: Red neuronal convolucional (Convolutional Neural Network).

CSV: Formato de datos separados por comas (Comma-Separated Values).

DBIR: Data Breach Investigations Report de Verizon.

DKIM: DomainKeys Identified Mail; firma criptográfica de los correos del dominio remitente.

DL: Deep Learning (aprendizaje profundo).

DMARC: Domain-based Message Authentication, Reporting and Conformance.

EML: Formato de archivo de correo electrónico estándar.

F1: Media armónica entre precisión y recall; métrica de evaluación de clasificadores.

GAN: Red generativa antagónica (Generative Adversarial Network).

GenAI: Inteligencia artificial generativa.

GPT: Generative Pre-trained Transformer.

GRU: Gated Recurrent Unit; tipo de red neuronal recurrente.

IA: Inteligencia artificial.

IC3: Internet Crime Complaint Center del FBI.

INCIBE: Instituto Nacional de Ciberseguridad de España.

LLM: Gran modelo de lenguaje (Large Language Model).

LSTM: Long Short-Term Memory; tipo de red neuronal recurrente.

M365: Microsoft 365; suite de productividad de Microsoft.

MFA: Autenticación multifactor (Multi-Factor Authentication).

ML: Machine Learning (aprendizaje automático).

MLP: Perceptrón multicapa (Multi-Layer Perceptron).

OAuth: Protocolo estándar de autorización delegada.

OSINT: Inteligencia de fuentes abiertas (Open Source Intelligence).

PhaaS: Phishing-as-a-Service; plataformas que ofrecen kits de phishing por suscripción.

QR: Código de respuesta rápida (Quick Response).

RCNN: Red neuronal convolucional recurrente (Recurrent CNN).

RF: Random Forest; algoritmo de aprendizaje por conjuntos de árboles de decisión.

SIEM: Security Information and Event Management; plataforma de correlación de eventos de seguridad.

SMS: Servicio de mensajes cortos (Short Message Service).

SPF: Sender Policy Framework; mecanismo de autenticación del remitente.

SVM: Máquina de vectores de soporte (Support Vector Machine).

TF-IDF: Term Frequency-Inverse Document Frequency; técnica de vectorización de texto.

UEBA: User and Entity Behavior Analytics; análisis del comportamiento de usuarios y entidades.

URL: Localizador uniforme de recursos (Uniform Resource Locator).

WHOIS: Protocolo de consulta de datos de registro de dominios.

XAI: Inteligencia artificial explicable (Explainable AI).

INDICE

Resumen	2
Abstract.....	3
Agradecimientos.....	4
Índice de figuras y tablas.....	5
Abreviaturas.....	6
Introducción	11
Marco teórico del phishing	12
Concepto y contexto actual	12
Fases de un ataque de phishing	12
Preparación y Recopilación de Información (Reconocimiento).....	13
Creación y Lanzamiento del Cebo (Setup y Ataque)	14
Interacción y Explotación (Colección Inicial)	15
Adquisición y Monetización de los Datos	15
Comparación crítica de modelos de fases.....	16
Puntos de intervención y controles	17
Tipos, clasificación y evolución de los ataques de phishing	17
Introducción a la clasificación	17
Clasificación según el vector de entrega	17
Clasificación según el grado de personalización y objetivo	18
Clasificación según la técnica de explotación	18
Evolución histórica resumida	19
Impacto actual (2025)	19
La ingeniería social como fundamento psicológico y sociotécnico del phishing	19
Principios psicológicos fundamentales explotados	20
Sesgos cognitivos específicos explotados	20
El impacto de la inteligencia artificial generativa en la ingeniería social (2023-2025).	20
Consecuencias organizativas y humanas	21
Implicaciones para la detección.....	21
Estado del arte en técnicas de detección y prevención de phishing basadas en Machine Learning y Deep Learning (2018-2025)	22
Evolución de los enfoques de detección.....	22
Técnicas clásicas de Machine Learning (representativas 2019-2022).....	22
Técnicas de Deep Learning y Transformers (2023-2025)	23
Enfoques híbridos y basados en comportamiento (UEBA).....	24
Limitaciones actuales de los enfoques.....	25
Conclusiones del estado del arte y hueco de investigación	25

Metodología y planificación	25
Enfoque metodológico	26
Planificación temporal del proyecto.....	26
Herramientas y tecnologías	27
Validación y control de calidad	27
Aspectos éticos y de protección de datos	28
Diseño e implementación de la aplicación web	29
Alcance del sistema	29
Cambios respecto a la propuesta inicial.....	29
Arquitectura general	30
Flujo de análisis	31
Sistema heurístico	32
Clasificador neuronal	32
Interfaz de usuario	33
Diseño modular.....	33
Limitaciones	34
Pruebas, resultados y discusión	35
Pruebas unitarias y de integración	35
Rendimiento y eficiencia de arranque	35
Diseño experimental y datasets	35
Resultados comparados	36
Comportamiento del sistema heurístico	37
Prueba funcional de la aplicación web	37
Evidencia visual del prototipo	38
Discusión	43
Conclusiones generales	43
Trabajo futuro	44
Glosario	45
Referencias.....	46
Anexos	47
Anexo A: Estructura del repositorio.....	47
Anexo B: Fragmentos de código fuente	48
B.1. Fachada del análisis heurístico (heurísticas.py).....	48
B.2. Servicio común de análisis (analysis_service.py)	48
B.3. Núcleo del clasificador neuronal (modelo_neural.py).....	49
Anexo C: Guía resumida de instalación y uso	50
C.1. Uso del prototipo	50

Anexo D: Lista de verificación de la validación	51
Anexo E: Resumen de arquitectura y evidencia	51

Introducción

La ciberseguridad se enfrenta a una amenaza persistente en el entorno digital: el phishing. La evolución de las campañas y el uso de inteligencia artificial generativa dificultan la detección basada únicamente en filtros y listas negras. Este escenario justifica estudiar aproximaciones que incorporen el contexto, el comportamiento y la intención del mensaje.

El objetivo general de este Trabajo Fin de Grado es diseñar, implementar y evaluar un prototipo de detección de phishing en correos electrónicos que combine un enfoque heurístico explicable con un clasificador automático basado en aprendizaje supervisado.

Para alcanzar este objetivo general se plantean los siguientes objetivos específicos:

- Analizar el fenómeno del phishing, sus fases, tipologías y evolución histórica.
- Estudiar los fundamentos psicológicos de la ingeniería social explotados por este tipo de ataques.
- Revisar el estado del arte en técnicas de detección basadas en machine learning y deep learning.
- Diseñar e implementar un sistema heurístico de señales explicables y un clasificador neuronal TF-IDF + MLP.
- Integrar la aplicación web con la API de Gmail y proporcionar alertas opcionales mediante Telegram.
- Evaluar el comportamiento del sistema mediante pruebas unitarias, de integración y funcionales.

La metodología seguida ha combinado una revisión bibliográfica de fuentes académicas y de industria con un desarrollo iterativo del prototipo. El sistema se ha implementado en Python con una arquitectura cliente-servidor: el navegador usa Streamlit como presentación, mientras Streamlit, la extensión y el monitor envían las entradas por HTTP al backend central. Solo el servidor normaliza, analiza, entrena y mantiene las versiones activas de los modelos. La validación se ha realizado mediante pruebas automatizadas, recorridos funcionales cliente-servidor y evaluación controlada del clasificador en español e inglés.

La memoria comienza con una introducción y un marco teórico dedicado al fenómeno del phishing, sus fases, tipologías y fundamentos psicológicos. A continuación, se revisa el estado del arte, se explica la metodología de trabajo y se describe el diseño de la aplicación web. Finalmente, se presentan las pruebas y resultados, las conclusiones, el glosario, las referencias y los anexos técnicos.

Marco teórico del phishing

Concepto y contexto actual

El phishing se ha consolidado como una amenaza persistente porque combina suplantación técnica e ingeniería social para inducir al usuario a revelar información, ejecutar una acción o transferir dinero. Su análisis exige considerar conjuntamente el canal, la identidad aparente, las cabeceras, el contenido y el contexto humano; ninguna señal aislada resulta suficiente en todos los casos.

El 2025 Data Breach Investigations Report de Verizon sitúa la participación humana en aproximadamente el 60 % de las brechas analizadas e identifica el phishing entre los vectores iniciales observados. Los informes del FBI muestran, además, que el fraude de correo corporativo puede ocasionar pérdidas económicas relevantes, aunque el impacto varía entre incidentes (Verizon, 2025; Federal Bureau of Investigation, 2024).

El phishing ha evolucionado desde campañas masivas hacia ataques dirigidos y mensajes asistidos por inteligencia artificial generativa. Estas herramientas pueden mejorar la fluidez y la adaptación contextual del texto, lo que reduce el valor de algunos indicadores lingüísticos tradicionales sin volverlos inútiles (Heiding et al., 2024).

En España, el Instituto Nacional de Ciberseguridad (INCIBE) gestionó 97.348 incidentes de ciberseguridad durante 2024, un 16,6 % más que en 2023. Dentro del fraude en línea, el phishing fue la tipología más frecuente, con 21.571 casos registrados (Instituto Nacional de Ciberseguridad, 2025).

Este crecimiento, combinado con la limitada efectividad de las soluciones tradicionales frente a ataques basados en inteligencia artificial, justifica la necesidad de desarrollar nuevas aproximaciones, objetivo central del presente Trabajo Fin de Grado.

Fases de un ataque de phishing

El phishing es un tipo de ataque cibernético que explota la confianza y el comportamiento humano para obtener información sensible, como credenciales de acceso o datos financieros. A diferencia de exploits técnicos directos, el phishing sigue un ciclo estructurado que puede desglosarse en fases secuenciales. Esta segmentación permite no solo entender el mecanismo del ataque, sino también identificar puntos de intervención para su detección y prevención.

Una de las anatomías más completas y actualizadas propone cuatro fases principales en el proceso de phishing: preparación y recopilación de información, creación y lanzamiento del cebo, interacción y explotación, y adquisición y monetización de los datos (Alkhalil et al., 2021). Este modelo integra perspectivas previas, como la "cadena de ataque" (kill chain) de tres fases —vector de amenaza, entrega y explotación— propuesta en análisis de seguridad email (GreatHorn, 2021; Hoxhunt, 2024). En esta memoria se adopta este enfoque, complementándolo con el marco MITRE ATT&CK (MITRE, 2025) y datos empíricos del Verizon DBIR 2025, que analiza 22.000 incidentes y 12.195 brechas confirmadas. A continuación, se detalla cada fase, con énfasis en sus componentes clave.

Figura 2.1: Diagrama de flujo swimlane de las fases de un ataque de phishing.



Fuente: Adaptado de Alkhalil et al. (2021) y MITRE ATT&CK (2025). Ilustra la progresión secuencial con énfasis en interacciones humano-técnicas.

Preparación y Recopilación de Información (Reconocimiento)

Esta fase inicial es fundamental para la personalización y efectividad del ataque. El atacante realiza una investigación exhaustiva sobre el objetivo, ya sea un individuo, organización o grupo amplio. Esto incluye la recopilación de datos públicos o filtrados, como correos electrónicos, perfiles en redes sociales, estructuras organizativas o vulnerabilidades técnicas conocidas.

- Actividades clave: Análisis de fuentes abiertas (OSINT, por sus siglas en inglés), como LinkedIn para spear phishing o bases de datos de brechas pasadas (p. ej., Have I Been Pwned). En ataques masivos, se usan bots para escanear listas de emails.
- Ejemplo: En un spear phishing corporativo, el atacante podría mapear la jerarquía de una empresa para targeting de ejecutivos (whaling).
- Duración y riesgos: La fase de reconocimiento puede prolongarse y no siempre deja señales visibles para la organización. El DBIR 2025 identifica el abuso de credenciales como uno de los vectores iniciales más frecuentes, lo que refuerza la necesidad de limitar la exposición pública y vigilar credenciales comprometidas (Verizon, 2025).

Interrumpir esta fase mediante políticas de privacidad y monitoreo de datos expuestos es un primer nivel de defensa.

Creación y Lanzamiento del Cebo (Setup y Ataque)

Una vez identificados los vectores de amenaza, el atacante diseña el "cebo" —el mensaje o vector de entrega— para que parezca legítimo y urgente. Aquí se aprovecha la ingeniería social para evadir filtros de spam y generar confianza.

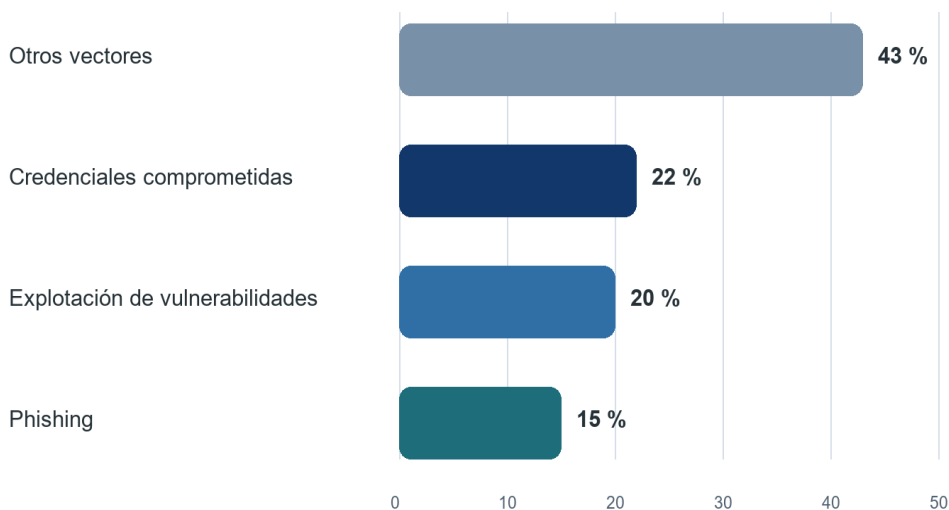
- Actividades clave: Creación de emails falsos, sitios web clonados (usando kits de phishing como Evilginx) o mensajes en otros canales (SMS para smishing, llamadas para vishing). Se incorporan elementos como logos robados, URLs acortadas o adjuntos maliciosos.
- Ejemplo: Un email simulando un banco que advierte de "actividad sospechosa" con un enlace a un sitio falso, o un QR code en un cartel físico que redirige a malware (quishing).
- Técnicas avanzadas: Uso de homógrafos (p. ej., "paypal.com" con caracteres cirílicos) o envenenamiento de DNS para redirigir tráfico (Imperva, 2023).

Esta fase es crítica porque la credibilidad del mensaje y su similitud con comunicaciones reales condicionan la interacción de la víctima. En el DBIR 2025, el phishing representó aproximadamente el 15 % de los vectores iniciales identificados en brechas, por detrás del abuso de credenciales y de la explotación de vulnerabilidades (Verizon, 2025).

Figura 2.2: Vectores iniciales en brechas analizadas por el DBIR 2025 (n=9.891).

Vectores iniciales en brechas

DBIR 2025 · n = 9.891 brechas analizadas



Fuente: Verizon, 2025 Data Breach Investigations Report.

Interacción y Explotación (Colección Inicial)

El objetivo es inducir una acción del usuario, como clicar un enlace, abrir un adjunto o ingresar datos. Aquí ocurre la "mordida" del anzuelo, donde la víctima interactúa con el payload malicioso.

- Actividades clave: El usuario puede ser redirigido a un sitio falso para capturar credenciales o inducido a abrir un adjunto malicioso. En ataques de varias etapas, un proxy adversary-in-the-middle (AiTM) puede interceptar tokens de sesión en tiempo real (Kroll, 2025).
- Ejemplo: En un ataque de clonación (clone phishing), se duplica un email legítimo previo y se reemplaza un adjunto seguro por uno infectado (Fortinet, 2025).
- Factores humanos: La urgencia, el miedo a perder acceso y la apariencia de legitimidad explotan sesgos cognitivos y pueden precipitar una decisión. La formación y la autenticación resistente al phishing reducen el riesgo, pero no eliminan la necesidad de controles técnicos.

Esta fase muestra la intersección entre tecnología y psicología: los controles técnicos deben complementarse con formación, autenticación resistente al phishing y mecanismos de detección. El DBIR 2025 sitúa la participación humana en el 60 % de las brechas analizadas (Verizon, 2025).

Adquisición y Monetización de los Datos

Con los datos en mano, el atacante los procesa y convierte en ganancia. Esta fase cierra el ciclo, pero puede extenderse a ataques secundarios.

- Actividades clave: Extracción de credenciales para accesos no autorizados, venta en dark web o uso directo (p. ej., transferencias fraudulentas). En BEC (Business Email Compromise), se roban fondos directamente.

- Ejemplo: En la campaña CorruptQR observada por Kroll, documentos de Office con cabeceras dañadas mostraban un código QR tras su reparación; al escanearlo, la víctima era dirigida a una página de phishing orientada al robo de credenciales corporativas y tokens de sesión (Kroll, 2025).
- Impacto: El coste económico depende del tipo de incidente y no debe atribuirse íntegramente al phishing. Como referencia general, IBM estimó en 4,44 millones de dólares el coste medio mundial de una brecha de datos en 2025, un 9 % menos que el año anterior (IBM, 2025).

Tabla 2.1: Indicadores de contexto del phishing y las brechas (2025).

Indicador	Medida	Valor	Fuente
Actividad observada	Ataques de phishing en Q2 2025	1.130.393	APWG
Acceso inicial	Phishing en brechas analizadas	≈ 15 %	Verizon DBIR
Factor humano	Brechas con participación humana	60 %	Verizon DBIR
Impacto general	Coste medio global por brecha	4,44 M USD	IBM

Comparación crítica de modelos de fases

Tabla 2.2: Comparativa de modelos

Autor/Año	Nº de fases	Nombres de las fases	Ventajas del modelo
GreatHorn (2023)	3	Vector → Delivery → Exploitation	Simplicidad operativa
Alkhalil et al. (2021)	4	Research → Creation → Interaction → Execution	Completo y académico
APWG (2024)	7	Planning → Research → Attack development → Attack → Collection → Fraud → Post-fraud	Detallado extremo
Modelo adoptado	4 + MITRE	Reconocimiento → Distribución → Interacción → Monetización + tácticas MITRE	Equilibrio rigor-calidad

Puntos de intervención y controles

Tabla 2.3: Controles por fase

Fase	Control preventivo	Control detectivo	Herramientas
Reconocimiento	Monitoreo OSINT	Alertas leaks	Have I Been Pwned
Distribución	Filtros email (SPF/DKIM/DMARC)	Análisis QR	Mimecast
Interacción	MFA resistente AiTM	Monitoreo clics	Microsoft Defender
Monetización	Detección anomalías	Análisis exfiltración	SIEM (Splunk, QRadar)

Este modelo de cuatro fases no es rígido; en ataques sofisticados, se solapan o iteran. Entenderlas es esencial para prototipos de detección, como los basados en ML que analizan patrones en la fase 2 (Alkhalil et al., 2021). Estas fases ilustran cómo el phishing ha evolucionado de ataques rudimentarios a campañas orquestadas, sentando las bases para discutir sus tipos y evolución histórica en las siguientes secciones.

Tipos, clasificación y evolución de los ataques de phishing

Introducción a la clasificación

Aunque el objetivo final del phishing es siempre el mismo —obtener información sensible o acceso no autorizado explotando la confianza humana—, la forma de llevarlo a cabo ha diversificado enormemente en los últimos años. En la actualidad, los ataques se clasifican principalmente según tres ejes: el canal o vector empleado, el grado de personalización del ataque y la técnica de explotación utilizada. Esta triple clasificación permite comprender mejor la evolución del fenómeno y anticipar las tendencias futuras.

Clasificación según el vector de entrega

El correo electrónico continúa siendo un canal central del phishing, aunque las campañas combinan cada vez más mensajes, páginas falsas, códigos QR y fraude de correo corporativo. En el segundo trimestre de 2025, APWG observó 1.130.393 ataques de phishing y documentó el uso de códigos QR contra 1.642 marcas (Anti-Phishing Working Group, 2025).

Los mensajes SMS (smishing), las llamadas telefónicas (vishing), las redes sociales y los códigos QR amplían los canales de entrega. Kroll observó en 2024 campañas que empleaban mensajes en redes sociales y documentos reparables que revelaban códigos QR, mientras que APWG dedicó en 2025 un seguimiento específico a los ataques mediante QR (Kroll, 2025; Anti-Phishing Working Group, 2025).

Otros vectores menos frecuentes pero en ascenso son el phishing a través de redes sociales (mensajes privados en LinkedIn, WhatsApp Business o Telegram), el envenenamiento de resultados de búsqueda (SEO poisoning) y el pharming o manipulación de resolución DNS.

Clasificación según el grado de personalización y objetivo

En función del nivel de targeting distinguimos varios tipos claramente diferenciados:

- Phishing masivo o genérico: campañas de gran volumen que reutilizan mensajes similares. Su coste reducido permite mantenerlas incluso cuando solo una fracción pequeña de los destinatarios interactúa.
- Spear-phishing: ataque dirigido contra una persona o un grupo reducido. Requiere investigación previa (OSINT) y mensajes adaptados al contexto, por lo que suele resultar más persuasivo que una campaña genérica.
- Whaling o CEO-fraud: variante del spear-phishing dirigida a altos ejecutivos o perfiles financieros. Puede producir pérdidas elevadas cuando deriva en fraude de correo corporativo, aunque el impacto varía ampliamente entre incidentes (Federal Bureau of Investigation, 2024).
- Clone phishing: se toma un correo legítimo previamente recibido por la víctima y se modifica un enlace o adjunto. Al conservar elementos de una conversación real, puede aumentar la confianza inicial.
- Adversary-in-the-Middle (AiTM): técnica que utiliza proxies inversos para interceptar la autenticación en tiempo real y capturar tokens de sesión incluso cuando la víctima emplea determinados métodos de autenticación multifactor. Constituye un riesgo relevante para servicios corporativos en la nube (Kroll, 2025).

Clasificación según la técnica de explotación

El tercer eje clasifica los ataques según la técnica de explotación, con independencia del canal de entrega o del nivel de personalización. Entre las tendencias recientes destacan el uso abusivo de inteligencia artificial generativa, la oferta de kits como servicio y las técnicas orientadas al robo de tokens:

1. Inteligencia artificial generativa: los modelos de lenguaje pueden ayudar a redactar mensajes fluidos, adaptar el tono al contexto y producir contenido web. Su resultado depende de la información y de las instrucciones disponibles, y no elimina otros indicios técnicos del ataque.
2. Phishing-as-a-Service (PhaaS): plataformas que empaquetan plantillas, alojamiento y mecanismos de evasión, reduciendo la barrera de entrada para campañas sofisticadas. Kroll relacionó CorruptQR con la plataforma ONNX y observó otros kits dirigidos a cuentas Microsoft 365 (Kroll, 2025).
3. Bypass de MFA mediante robo de tokens: los proxies adversary-in-the-middle pueden capturar cookies o tokens de sesión después de una autenticación válida. Por ello se recomiendan métodos resistentes al phishing, como FIDO, junto con controles de acceso condicional (Kroll, 2025).

Evolución histórica resumida

El phishing ha pasado por varias etapas claramente diferenciadas:

- 1995-2003: orígenes en AOL y primeros ataques masivos a PayPal.
- 2004-2010: aparición de kits (Rock Phish, Avalanche) y profesionalización.
- 2011-2016: auge del spear-phishing y del ransomware entregado vía phishing.
- 2016-2020: explosión del Business Email Compromise (BEC).
- 2020-2022: pandemia y boom de smishing/vishing.
- 2022-2025: expansión de la IA generativa, AiTM, quishing y PhaaS, con una mayor combinación de canales, automatización y servicios criminales.

En tres décadas, el phishing ha pasado de campañas rudimentarias a operaciones criminales organizadas que combinan automatización, ingeniería social y servicios comercializados en mercados clandestinos.

Impacto actual (2025)

El volumen sigue siendo elevado: APWG observó 1.130.393 ataques de phishing en el segundo trimestre de 2025. En España, INCIBE gestionó 97.348 incidentes de ciberseguridad durante 2024, incluidos 21.571 casos de phishing. Como indicador económico general, no específico de phishing, IBM situó el coste medio mundial de una brecha en 4,44 millones de dólares en 2025 (Anti-Phishing Working Group, 2025; Instituto Nacional de Ciberseguridad, 2025; IBM, 2025).

Este panorama justifica complementar los filtros de correo y las listas negras con sistemas capaces de analizar el contexto, el comportamiento y la intención del mensaje, objetivo principal del presente trabajo.

La ingeniería social como fundamento psicológico y sociotécnico del phishing

Aunque el phishing se manifiesta a través de medios técnicos —correo electrónico, SMS, sitios web falsos o códigos QR—, su eficacia depende en gran medida de la explotación de factores cognitivos, emocionales y sociales. Por ello, la ingeniería social constituye un componente central del fenómeno (Hadnagy, 2018).

Principios psicológicos fundamentales explotados

Los atacantes diseñan sus campañas apoyándose en principios psicológicos bien documentados, muchos de ellos identificados originalmente por Robert Cialdini (1984) y posteriormente validados en el ámbito de la ciberseguridad:

Tabla 2.4: Aplicación de los principios de Cialdini en campañas de phishing.

Principio	Aplicación habitual en phishing	Riesgo para la víctima
Autoridad	Suplantación de bancos, administraciones, dirección o proveedores	Reduce el cuestionamiento de la solicitud
Urgencia / escasez	Plazos breves, bloqueo de cuentas o falsas oportunidades	Favorece decisiones precipitadas
Reciprocidad	Regalos, descuentos o supuestas actualizaciones gratuitas	Normaliza la primera interacción
Compromiso y coherencia	Secuencias que comienzan con una acción pequeña	Aumenta la probabilidad de continuar
Prueba social	Avisos sobre acciones atribuidas a otros usuarios	Hace que la petición parezca habitual
Simpatía	Identidades conocidas, tono cercano o afinidad profesional	Incrementa la confianza inicial

Sesgos cognitivos específicos explotados

Más allá de los principios de influencia, los atacantes aprovechan sesgos cognitivos automáticos (Kahneman, 2011; Tversky & Kahneman, 1974) que operan a nivel Sistema 1 (rápido, intuitivo):

- Sesgo de anclaje: el usuario ancla su percepción en el remitente aparentemente legítimo.
- Efecto de mera exposición: cuanto más parecido sea el dominio o el diseño al original, mayor confianza genera.
- Aversión a la pérdida (loss aversion): el miedo a perder el acceso a una cuenta o sufrir una penalización puede pesar más que una ganancia equivalente y precipitar la respuesta.
- Fatiga de decisión y sobrecarga cognitiva: en entornos laborales con alto volumen de correo, la atención selectiva colapsa (GreatHorn, 2021).

La combinación de urgencia, autoridad y prueba social puede elevar la persuasión del mensaje. Este efecto depende del contexto, de la experiencia previa y de las diferencias individuales, por lo que no debe expresarse como una tasa universal de éxito (Heiding et al., 2024).

El impacto de la inteligencia artificial generativa en la ingeniería social (2023-2025)

La disponibilidad de modelos de lenguaje grandes (LLM) ha introducido cambios cualitativos en la preparación de mensajes de ingeniería social:

- Reducción de indicadores lingüísticos tradicionales: los modelos generativos pueden producir mensajes fluidos y disminuir errores gramaticales o construcciones extrañas que antes facilitaban la detección (Heiding et al., 2024).
- Personalización contextual: los modelos de lenguaje pueden adaptar el tono, incorporar información pública y reproducir terminología propia de un sector, aunque la calidad depende de los datos y de las instrucciones disponibles.
- Generación dinámica de narrativas: las herramientas generativas pueden producir y adaptar textos con rapidez. Su uso abusivo reduce el esfuerzo de personalización, aunque no garantiza que el mensaje resulte creíble ni eficaz.

Los experimentos de Heiding et al. (2024) muestran que los modelos de lenguaje pueden producir correos de phishing competitivos frente a mensajes redactados por personas y que la incorporación de principios de persuasión modifica su eficacia. Los resultados corresponden a un diseño experimental concreto y no constituyen una tasa universal de éxito.

Consecuencias organizativas y humanas

El Verizon DBIR 2025 sitúa la participación humana en el 60 % de las brechas analizadas. En España, el 33 % de los usuarios que contactaron con la línea 017 indicó haber recibido algún intento de phishing, vishing o smishing durante 2024 (Verizon, 2025; Instituto Nacional de Ciberseguridad, 2025).

Este factor humano no es estático: la fatiga por alerta, la sobrecarga de correo y la normalización del trabajo remoto reducen la atención sostenida. Los informes de industria señalan además que determinados perfiles, como finanzas, recursos humanos o dirección, reciben un volumen desproporcionado de ataques dirigidos (Proofpoint, 2025).

Implicaciones para la detección

La dimensión psicológica del phishing implica que una solución eficaz debe combinar el análisis técnico con señales del contexto humano. Los sistemas basados únicamente en características estáticas de URL o contenido pueden degradarse cuando cambian las campañas o se emplea texto generado (Alghenaim et al., 2025; Heiding et al., 2024).

Por tanto, entre las líneas de evolución de la detección se encuentran los modelos que incorporan:

- Análisis semántico profundo del texto (transformers, LLMs).
- Modelado del comportamiento del usuario (UEBA).
- Detección de intención y manipulación psicológica mediante NLP avanzado.

Este será precisamente el fundamento teórico sobre el que se construya el estado del arte y el prototipo propuesto en las siguientes secciones.

Estado del arte en técnicas de detección y prevención de phishing basadas en Machine Learning y Deep Learning (2018-2025)

La detección de phishing ha evolucionado desde reglas y listas de bloqueo hacia modelos de machine learning y deep learning capaces de combinar rasgos de URL, contenido, cabeceras y contexto. Las cifras publicadas no son directamente comparables porque varían el corpus, el equilibrio de clases, la partición de datos y la definición de phishing; por ello, esta revisión prioriza las aportaciones y limitaciones de cada enfoque sobre una clasificación basada únicamente en accuracy (Thakur et al., 2023; Alghenaim et al., 2025).

La sección revisa trabajos académicos e informes técnicos entre 2018 y 2025. Se organiza en enfoques clásicos, deep learning, transformers, métodos híbridos y limitaciones, y utiliza las revisiones citadas para contextualizar estudios representativos sin presentar sus métricas como si procedieran de un único benchmark.

Evolución de los enfoques de detección

La progresión se divide en tres fases principales, alineadas con la madurez tecnológica:

1. 2019-2021: Enfoques clásicos basados en características manuales. Random Forest, SVM y otros clasificadores trabajan con URL, HTML, cabeceras y variables léxicas. Son rápidos y relativamente explicables, pero dependen de rasgos que pueden envejecer cuando cambian las tácticas.
2. 2022-2023: Consolidación del deep learning. CNN, LSTM y modelos preentrenados reducen parte de la ingeniería manual y aprenden patrones en URLs o texto. Su comparación sigue limitada por corpus distintos y por evaluaciones realizadas en entornos controlados.
3. 2024-2025: Transformers y modelos de lenguaje. Los trabajos recientes estudian clasificación zero-shot, ajuste fino, prompt engineering y explicaciones generadas por LLM. El potencial es alto, pero la precisión, la fidelidad de las explicaciones, el coste y la privacidad no mejoran siempre de forma simultánea (Koide et al., 2024; Trad & Chehab, 2024; Kuikel et al., 2025).

Esta evolución refleja el paso de reglas fijas a modelos que aprenden representaciones más ricas. No elimina la utilidad de los enfoques clásicos: en despliegues locales, la velocidad, la trazabilidad y el coste siguen siendo criterios relevantes.

Técnicas clásicas de Machine Learning (representativas 2019-2022)

Estos métodos priorizan eficiencia computacional y explicabilidad, pero requieren feature engineering manual.

Tabla 3.1: Comparativa de técnicas clásicas de machine learning.

Año	Trabajo	Enfoque	Objeto	Aportación	Limitación
2018	Hutchinson et al.	Random Forest	Sitios web	Muestra la utilidad de rasgos de URL y página	Depende de variables diseñadas manualmente

Año	Trabajo	Enfoque	Objeto	Aportación	Limitación
2020	Yerima y Alzaylaee	Red convolucional	URLs y páginas	Aprende representaciones con menos ingeniería manual	No cubre por sí sola la semántica completa del correo
2020	AlEroud y Karabatis	Red adversarial	Evasión de detectores	Estudia robustez frente a ejemplos generados	La generalización fuera del laboratorio es difícil
2022	Sánchez-Paniagua et al.	Dataset multipropósito	Sitios web	Facilita comparaciones reproducibles	No representa todos los canales de phishing

(Fuentes: Hutchinson et al., 2018; Yerima & Alzaylaee, 2020; AlEroud & Karabatis, 2020; Sánchez-Paniagua et al., 2022).

Los enfoques clásicos extraen características de URLs, páginas, cabeceras o texto y las proporcionan a clasificadores supervisados. Los modelos que aprenden representaciones directamente de la URL reducen parte de esa ingeniería manual (Le et al., 2018), pero su rendimiento también puede caer cuando cambia la distribución o el atacante modifica los rasgos esperados.

Yerima y Alzaylaee (2020) aplicaron redes convolucionales a la detección de phishing, mientras que AlEroud y Karabatis (2020) estudiaron ataques adversariales contra detectores. En conjunto, estos trabajos desplazan el foco desde la precisión estática hacia la capacidad de aprender representaciones y resistir evasiones.

Sánchez-Paniagua et al. (2022) publicaron un dataset multipropósito y características de tecnologías web para facilitar comparaciones reproducibles. La limitación común de esta generación es que un buen resultado sobre páginas o URLs no demuestra por sí solo el comportamiento ante correos nuevos, idiomas distintos o campañas generadas por IA.

Técnicas de Deep Learning y Transformers (2023-2025)

Los avances recientes enfatizan multimodalidad y zero-shot, con énfasis en LLMs para detección semántica.

Tabla 3.2: Comparativa de técnicas de deep learning y transformers.

Año	Trabajo	Enfoque	Objeto	Aportación	Limitación
2023	Thakur et al.	Revisión sistemática	Correo	Sintetiza arquitecturas de deep learning	Los datasets y métricas son heterogéneos
2023	Wang et al.	Modelo preentrenado	URLs	Explora transferencia y detección a gran escala	Se centra en la URL, no en el mensaje completo
2024	Altwaijry et al.	Comparativa DL	Correo	Compara varias arquitecturas sobre una misma tarea	El resultado depende del corpus evaluado
2024	Koide et al.	LLM zero-shot	Correo	Evalúa detección sin ajuste específico	Coste, privacidad y reproducibilidad

Año	Trabajo	Enfoque	Objeto	Aportación	Limitación
2024	Rojas-Galeano	LLM zero-shot	Spam	Analiza transferencia entre modelos preentrenados	Spam y phishing no son categorías equivalentes
2024	Uddin et al.	Transformer explicable	Correo	Relaciona predicción con términos relevantes	Preprint pendiente de validación más amplia
2025	Kuikel et al.	LLM + explicaciones	Correo	Compara precisión y alineación de explicaciones	Existe un compromiso entre acierto y fidelidad
2025	Alghenaim et al.	Revisión del estado del arte	Detección con IA	Resume tendencias, riesgos y líneas abiertas	No es un clasificador evaluado de forma original

(Fuentes: Thakur et al., 2023; Wang et al., 2023; Altwaijry et al., 2024; Koide et al., 2024; Rojas-Galeano, 2024; Uddin et al., 2024; Kuikel et al., 2025; Alghenaim et al., 2025).

Los trabajos recientes amplían el objeto de análisis, pero difieren en corpus, tarea y protocolo de evaluación. La tabla resume su aportación principal sin mezclar resultados que no proceden del mismo benchmark.

Las revisiones de Thakur et al. (2023) y Alghenaim et al. (2025) describen una transición hacia redes profundas, modelos preentrenados y combinaciones de texto, URL y metadatos. Este avance reduce parte de la ingeniería manual, aunque mantiene problemas de comparabilidad y deriva temporal.

Altwaijry et al. (2024) compararon modelos de deep learning para correo y Koide et al. (2024) exploraron un detector basado en modelos de lenguaje. Ambos trabajos muestran el interés por incorporar contexto semántico, pero no justifican trasladar una cifra de accuracy a otros corpus o entornos.

Rojas-Galeano (2024) estudió clasificación zero-shot de spam, una tarea próxima pero no idéntica al phishing. Uddin et al. (2024) abordaron la explicabilidad con transformers y Kuikel et al. (2025) observaron que una mejor alineación entre predicción y explicación no implica necesariamente mayor precisión. Esta tensión es central para sistemas de apoyo a la decisión.

Enfoques híbridos y basados en comportamiento (UEBA)

Los enfoques híbridos combinan señales estructurales, contenido y comportamiento para aportar contexto que una única fuente no contiene. La integración puede mejorar cobertura, pero aumenta la complejidad de datos, despliegue y evaluación.

- Híbridos semánticos e ingeniería social: combinan representaciones del texto con señales de urgencia, autoridad o solicitud de credenciales. Su principal reto es demostrar que esas señales generalizan fuera del corpus de entrenamiento.
- Multimodal: fusiona texto, URL, imágenes y metadatos. Puede detectar señales complementarias, aunque exige datasets alineados y eleva el coste de entrenamiento e inferencia.

- Zero-shot: utiliza modelos preentrenados sin ajuste específico. Es útil para explorar ataques emergentes, pero introduce variabilidad, coste y posibles transferencias de datos a servicios externos.
- Prevención proactiva: UEBA analiza cambios de comportamiento y anomalías de acceso. Complementa la inspección del mensaje, aunque requiere telemetría organizativa que queda fuera del alcance de este prototipo.

La literatura reciente presenta los métodos híbridos como una línea prometedora, pero subraya la necesidad de evaluar cada componente y evitar que la complejidad oculte qué señal sostiene la decisión (Alghenaim et al., 2025; Popescul y Radu, 2025).

Limitaciones actuales de los enfoques

Aunque distintos trabajos publican resultados elevados en sus propios corpus, persisten desafíos que impiden compararlos directamente:

1. Datasets desfasados: muchos corpus no reflejan campañas recientes, IA generativa, quishing ni deriva de dominios. Un resultado interno elevado puede degradarse ante ataques nuevos.
2. Coste y latencia: los modelos grandes exigen más memoria, cómputo o llamadas a servicios externos; los modelos locales ligeros ofrecen mayor control y reproducibilidad.
3. Multilingüismo y sesgo: el predominio de corpus en inglés dificulta trasladar resultados al español y a contextos culturales diferentes.
4. Robustez adversarial: homógrafos, ofuscación, cambios de plantilla y texto generado pueden eludir patrones aprendidos. Pocos trabajos evalúan de forma comparable estas evasiones.
5. Explicabilidad: una explicación generada no garantiza fidelidad al proceso de decisión. Debe distinguirse entre texto persuasivo y evidencia verificable del modelo.

La revisión bibliométrica de Popescul y Radu (2025) confirma el crecimiento de ML, deep learning y modelos híbridos, y señala como líneas abiertas la adaptación a nuevos vectores, la robustez y la integración organizativa.

Conclusiones del estado del arte y hueco de investigación

La literatura reciente sitúa los transformers y los LLM entre las líneas activas de investigación, pero advierte que los resultados de entornos controlados no son directamente comparables. Persisten tres huecos relevantes:

1. Falta de fine-tuning open-source para español/multilingüe
2. Escasa integración de detección psicológica (Cialdini) en LLMs
3. Evaluación limitada contra ataques GenAI en tiempo real.

Metodología y planificación

Este capítulo describe el enfoque metodológico seguido para el desarrollo del prototipo, así como la planificación temporal del proyecto y las herramientas empleadas. El trabajo se ha desarrollado a lo largo del curso académico 2025-2026, distribuyendo las actividades de análisis y documentación en el primer cuatrimestre y las de implementación y validación en el segundo.

Enfoque metodológico

El proyecto se ha desarrollado siguiendo una metodología iterativa e incremental, combinada con una revisión bibliográfica continua. Cada iteración añade una capacidad nueva al sistema y se cierra con la ejecución de la suite de pruebas, de modo que el prototipo permanece en un estado funcional durante todo el proceso.

La fase inicial, desarrollada entre octubre y diciembre de 2025, se dedicó al estudio del contexto del problema y a la revisión de la literatura académica (IEEE, arXiv y Springer) y de los informes de industria citados en el estado del arte. Esta etapa permitió identificar las técnicas de detección más relevantes, delimitar el hueco de investigación y definir los objetivos específicos y el alcance del sistema.

Una vez consolidado el marco teórico, se adoptó un enfoque de desarrollo en espiral adaptado al tamaño del trabajo, llevado a cabo entre febrero y junio de 2026. Primero se construyó un núcleo heurístico explicable, después se añadió el clasificador neuronal TF-IDF + MLP y, a continuación, se separaron el backend HTTP central y los clientes Streamlit, extensión y monitor. Gmail aporta mensajes autorizados y Telegram actúa como salida opcional; ambos se coordinan alrededor del contrato del servidor. Cada avance se cerró con la ejecución de la suite de pruebas, manteniendo el prototipo en un estado funcional durante todo el proceso.

Durante el desarrollo se emplearon herramientas de inteligencia artificial generativa como apoyo para consultas técnicas puntuales, diagnóstico y resolución de errores, revisión de fragmentos de código, asistencia en el CSS de la interfaz y revisión de documentación. Antes de incorporar los cambios se realizaron las comprobaciones correspondientes; las decisiones técnicas, la interpretación de los resultados y la responsabilidad final corresponden al autor. Estas herramientas no se utilizaron como fuente académica: los contenidos teóricos se contrastaron y citaron mediante sus fuentes originales.

La gestión del proyecto se apoyó en el control de versiones Git y en la plataforma GitHub como repositorio remoto. El desarrollo se organizó por funcionalidades y cada hito se validó mediante pruebas automatizadas antes de integrarse en la versión de trabajo, con especial atención a las funciones críticas de detección, normalización y persistencia de modelos.

Planificación temporal del proyecto

La planificación se distribuyó en siete fases repartidas a lo largo del curso académico. La tabla siguiente resume la planificación temporal, las actividades principales de cada fase y el entregable asociado. Las fases de análisis y documentación se concentraron en el primer cuatrimestre, mientras que la implementación del prototipo y su validación se desarrollaron en el segundo.

Tabla 4.1: Planificación temporal del proyecto.

Fase	Periodo	Actividades principales	Entregable
Fase 1: Análisis del contexto y estado del arte	Octubre - diciembre 2025	Revisión bibliográfica, análisis del phishing, sus tipologías, la ingeniería social y las técnicas de detección.	Marco teórico y estado del arte
Fase 2: Diseño funcional y modular	Enero - febrero 2026	Definición del alcance de la aplicación web, separación de responsabilidades y selección de tecnologías.	Especificación funcional y diseño modular
Fase 3: Motor heurístico	Febrero - marzo 2026	Implementación del analizador, señales de cabeceras, contenido, URLs y HTML, y cálculo del riesgo.	Núcleo heurístico con pruebas

Fase	Periodo	Actividades principales	Entregable
Fase 4: Clasificador neuronal	Marzo - abril 2026	Pipeline TF-IDF + MLP, entrenamiento en español e inglés y comparación de configuraciones.	Modelos neuronales entrenables
Fase 5: Aplicación web	Abril - mayo 2026	Desarrollo de las vistas Streamlit de inicio, configuración, detección, monitorización y entrenamiento.	Aplicación web funcional
Fase 6: Integraciones y validación	Mayo - junio 2026	Integración OAuth con Gmail, alertas de Telegram, pruebas automatizadas y evaluación funcional.	Prototipo integrado y resultados
Fase 7: Documentación final	Junio 2026	Validación final, redacción de la memoria y revisión de la documentación.	Memoria del TFG

La carga de trabajo del segundo cuatrimestre se concentró en el motor heurístico, el clasificador neuronal y el backend central, que reúnen la lógica del prototipo. Después se adaptaron Streamlit, Gmail, la extensión y el monitor como clientes del mismo contrato HTTP. La extensión llama directamente al servidor; el proceso del puerto 8765 se conserva únicamente como proxy de compatibilidad, sin reglas ni modelos propios.

Herramientas y tecnologías

La tabla siguiente recoge las principales herramientas y tecnologías utilizadas durante el desarrollo, junto con el propósito de cada una en el proyecto.

Tabla 4.2: Herramientas y tecnologías utilizadas.

Herramienta / tecnología	Propósito en el proyecto
Python 3	Lenguaje de programación principal del prototipo.
Streamlit	Interfaz web con vistas de inicio, configuración, detección, monitorización y entrenamiento.
BeautifulSoup4	Procesamiento del HTML incluido en los correos y extracción de señales estructurales.
NumPy	Operaciones numéricas utilizadas durante el entrenamiento y la evaluación.
scikit-learn	Vectorizador TF-IDF, clasificador MLPClassifier y métricas de evaluación.
joblib	Persistencia en disco de los modelos neuronales entrenados.
langdetect	Selección del modelo en español o inglés según el contenido del correo.
Gmail API (OAuth)	Importación autorizada de correos desde la bandeja del usuario.
Bot API de Telegram	Envío opcional de alertas desde la monitorización de Gmail.
unittest	Pruebas unitarias y de integración de los componentes del prototipo.
Git y GitHub	Control de versiones, trazabilidad y respaldo del desarrollo.

Validación y control de calidad

La validación del sistema se realiza en seis niveles complementarios: 94 pruebas Python de componentes e integración, 2 recorridos reales con Chromium, un benchmark reproducible, una calibración bilingüe de 40 casos, 16 EML locales reservados y un diagnóstico de 1.528 textos externos DIFrauD. Un recorrido levanta cliente web y backend separados; otro prueba

Gmail, HTTP y Telegram de extremo a extremo con dobles externos. Los EML son sintéticos y DIFrauD conserva riesgo de solapamiento con fuentes de entrenamiento; ninguna cifra se extrapola a producción.

En segundo lugar, la aplicación permite evaluar el clasificador neuronal con un CSV de prueba independiente, separado del conjunto de entrenamiento. La interfaz calcula exactitud, precisión, recall, F1, accuracy balanceada y la matriz de confusión; no se fija una partición automática 70/30 ni se presentan resultados estadísticos sin indicar el conjunto utilizado. En tercer lugar, se comprobaron los flujos funcionales de la aplicación web: análisis de texto pegado y archivos .eml, selección del modelo por idioma, importación mediante Gmail y navegación entre las vistas de detección y entrenamiento. Los resultados reproducibles se presentan en la sección de pruebas y discusión.

Aspectos éticos y de protección de datos

El tratamiento de los correos electrónicos analizados por el prototipo se plantea conforme al Reglamento General de Protección de Datos (RGPD, Reglamento (UE) 2016/679). El sistema minimiza la información procesada: solo extrae los metadatos y las señales necesarias para la clasificación (cabeceras, remitente, asunto, enlaces y estructura HTML), sin almacenar de forma persistente el contenido íntegro de los mensajes, las credenciales de acceso ni los datos personales de los remitentes.

El acceso a la API de Gmail se realiza mediante OAuth, delegando en Google la autenticación del usuario y evitando el manejo de contraseñas en la aplicación. El prototipo está diseñado para entornos controlados y académicos, con datos de demostración o con el consentimiento explícito del usuario. Los tokens y credenciales se almacenan localmente y quedan excluidos del repositorio. Antes de publicar la aplicación para varios usuarios sería necesario incorporar autenticación, gestión segura de secretos y una política explícita de conservación de datos.

Diseño e implementación de la aplicación web

Tras el análisis teórico de las técnicas de phishing y de los principales enfoques de detección existentes, se ha desarrollado un prototipo funcional orientado al análisis de correos electrónicos sospechosos. El objetivo del sistema no es sustituir a una solución empresarial completa de seguridad, sino demostrar cómo puede combinarse un enfoque heurístico explicable con un clasificador automático basado en aprendizaje supervisado.

El prototipo se centra en el análisis de mensajes desde varios clientes. El origen puede ser texto, EML, Gmail, la extensión o JSON. El cliente transporta la entrada y el backend central extrae cabeceras, remitente, asunto, cuerpo, HTML, enlaces, anclas y adjuntos para generar el riesgo heurístico, neuronal o combinado. La interfaz recibe un contrato ya calculado y se limita a presentarlo.

Alcance del sistema

El alcance del prototipo se ha delimitado al análisis estático del correo electrónico y de las URLs contenidas en él. El sistema no navega activamente hacia las páginas enlazadas, no descarga contenido externo ni consulta servicios de reputación en tiempo real. La API, la extensión, Streamlit y los procesos de monitorización se mantienen en el entorno local y no convierten el proyecto en un servicio remoto multiusuario.

No obstante, la arquitectura queda preparada para ampliaciones futuras. Podrían añadirse consultas a listas negras públicas, reputación de dominios, validación de certificados TLS, análisis dinámico de páginas o integración con sistemas corporativos de alerta. El prototipo no se plantea, por tanto, como un producto cerrado, sino como una base funcional y extensible.

Cambios respecto a la propuesta inicial

La propuesta de octubre de 2025 contemplaba un prototipo con reglas, servicios externos de reputación, comprobación de certificados y la posible utilización de TensorFlow. Durante el desarrollo se ajustó el alcance para priorizar privacidad, seguridad de la evaluación, reproducibilidad local y una arquitectura cliente-servidor con un único modelo compartido.

Tabla 5.1: Cambios respecto a la propuesta inicial.

Elemento	Propuesta	Implementación final	Justificación
Aprendizaje	scikit-learn y TensorFlow	TF-IDF + MLPClassifier de scikit-learn	Pipeline ligero, reproducible y suficiente para el alcance; TensorFlow no era necesario.
Reputación	Listas negras y servicios externos	Listas y señales locales; sin consultas online	Evita enviar URLs sensibles, depender de cuotas o visitar infraestructura potencialmente maliciosa.
Certificados	Comprobación de certificados digitales	Sin conexión al destino ni validación TLS	Un EML no acredita el certificado actual del enlace; la comprobación activa queda como trabajo futuro aislado.
Arquitectura	Interfaz sencilla del prototipo	Clientes HTTP y backend central obligatorio	Una actualización del modelo se aplica a web, extensión y monitor sin distribuir copias.

Estos cambios no alteran el objetivo académico de estudiar, implementar y evaluar la detección de phishing. Delimitan qué se demuestra en el prototipo y qué capacidades necesitan infraestructura y controles adicionales.

Arquitectura general

La solución adopta una arquitectura cliente-servidor. El navegador se comunica con Streamlit, que actúa como capa de presentación y cliente HTTP del backend central; la extensión y el monitor consumen esa misma API. Solo backend_server.py normaliza, analiza, entrena y mantiene los modelos activos. Cada instalación conserva credenciales y preferencias en runtime/client, mientras el backend es propietario de los ajustes y modelos de runtime/server; la interfaz administra los valores centrales por API, no mediante acceso directo al disco. En la configuración académica ambos lados se ejecutan en el mismo equipo y sobre loopback, pero son procesos y responsabilidades independientes. Para una demostración en la red local puede exponerse temporalmente solo Streamlit en 0.0.0.0:8501; el backend permanece en 127.0.0.1:8766 y no se publica.

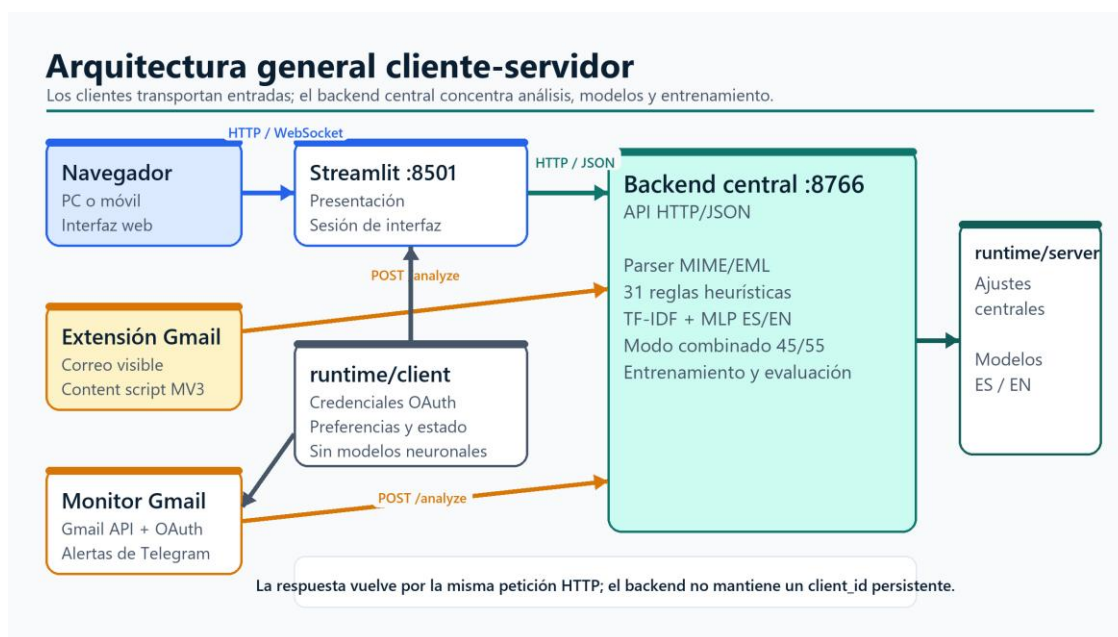


Figura 5.1: Arquitectura general cliente-servidor del prototipo. Fuente: elaboración propia.

En cuanto a concurrencia, el backend usa ThreadingHTTPServer y crea un hilo por petición, sin un máximo cerrado en el código. En este equipo se repitieron pruebas de 8 y 16 análisis simultáneos sin fallos; con 16, el percentil 95 de latencia quedó entre 0,58 y 0,66 s, mientras que a 32 aparecieron errores. Por ello se recomiendan 4-8 solicitudes concurrentes para la configuración académica y se presenta 16 solo como máximo comprobado localmente, no como capacidad de producción. Entrenamiento y borrado de modelos se serializan mediante un bloque.

A nivel interno, backend_client.py implementa el transporte común de los clientes y http_api.py define las rutas, incluidas las operaciones administrativas de ajustes. backend_service.py concentra análisis, configuración central, datasets, evaluación, entrenamiento, metadatos y activación de versiones. runtime_paths.py impone las rutas separadas: credenciales, token y estado en runtime/client; configuración y modelos en runtime/server. Dentro del servidor, analysis_service.py coordina los modos y la caché lingüística; analizador_email.py normaliza EML; signal_builder.py, scorer.py y explanations.py construyen el resultado; modelo_neural.py encapsula TF-IDF + MLP. Las vistas Streamlit no importan almacenamiento de modelos ni ejecutan inferencia.

Las reglas heurísticas se han dividido en módulos especializados: header_signals.py para cabeceras y autenticación, content_signals.py para contenido textual y adjuntos,

html_signals.py para estructuras HTML sospechosas y url_utils.py para extracción y análisis de URLs y dominios. La detección del idioma está centralizada en idioma.py, de modo que la aplicación y la monitorización seleccionan el modelo neuronal con el mismo criterio. Esta división mejora la mantenibilidad y facilita añadir nuevas reglas sin modificar el núcleo del analizador.

Flujo de análisis

El flujo general comienza con la entrada del correo. Si el usuario sube un archivo .eml, el parser extrae remitente, destinatario, asunto, cuerpo en texto plano, cuerpo HTML, cabeceras, enlaces, anclas y adjuntos. Si el usuario pega el contenido manualmente, se genera una representación simplificada. Cuando la entrada procede de Gmail, la propia aplicación obtiene los mensajes autorizados mediante OAuth y los adapta al mismo formato interno.

A continuación, la entrada se normaliza en una representación interna común (CorreoAnalizado), de modo que las reglas heurísticas no dependen del formato original del mensaje.

Una vez normalizado el correo, SignalBuilder ejecuta 31 reglas y produce un diccionario estable de señales. Además de cabeceras, autenticación, URLs, HTML y adjuntos, incorpora tres patrones bilingües para BEC sin enlace: cambio de datos bancarios, orden urgente de transferencia y suplantación de un directivo bajo aislamiento o confidencialidad. Si el modo lo requiere, el backend detecta el idioma y reutiliza el modelo neuronal central correspondiente.

RiskScorer asigna una ponderación a cada señal activa y calcula una puntuación final entre 0 y 100. Si la puntuación supera el umbral configurado, el mensaje se clasifica como phishing probable. ExplanationBuilder transforma las señales en explicaciones comprensibles. El backend central devuelve ese resultado por HTTP a Streamlit, la extensión o el monitor; los clientes solo lo presentan o, en el caso del monitor, pueden generar una alerta de Telegram y registrar el identificador para evitar duplicados.

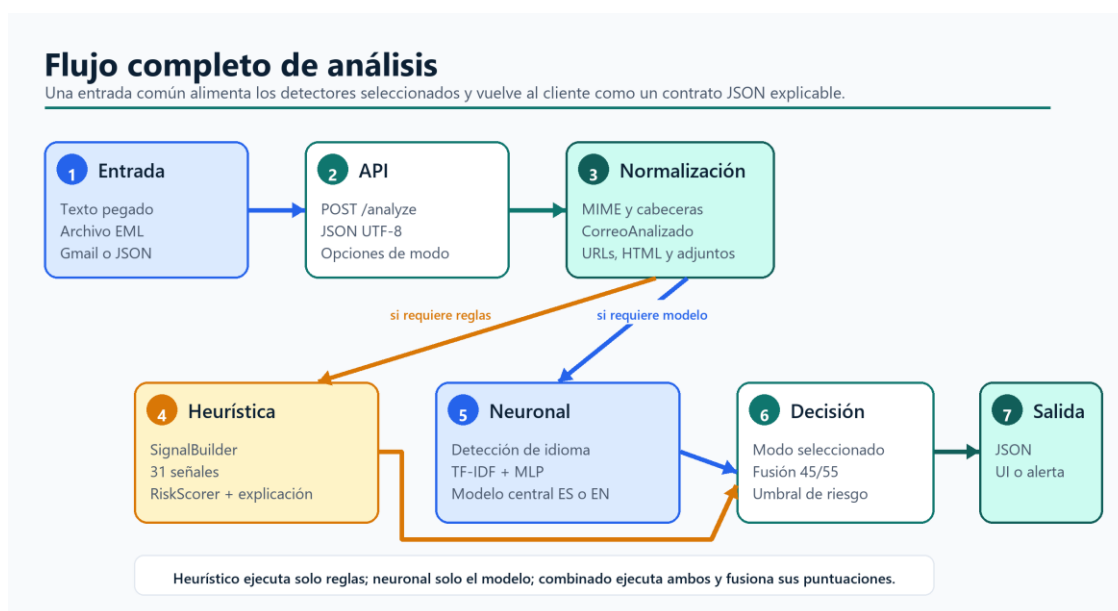


Figura 5.2: Flujo de análisis desde la entrada hasta la respuesta del backend. Fuente: elaboración propia.

Sistema heurístico

El sistema heurístico constituye la parte más explicable del prototipo. Está formado por un conjunto de reglas diseñadas a partir de patrones habituales en correos de phishing. Estas reglas se agrupan en varias categorías.

En primer lugar, se analizan las cabeceras del correo. El sistema interpreta los resultados que los servidores de correo ya han escrito en Received-SPF, Authentication-Results y ARC-Authentication-Results, y detecta estados fail, softfail o errores de SPF, DKIM y DMARC. También revisa incoherencias entre From, Reply-To y Return-Path y si una cabecera DKIM-Signature presente está mal formada. No realiza consultas DNS, no recupera claves públicas y no verifica criptográficamente SPF, DKIM o DMARC; por tanto, es una interpretación pasiva de cabeceras, no una validación completa de autenticación.

En segundo lugar, se analizan los enlaces. El sistema extrae URLs del texto y del HTML, comprueba la presencia de dominios en listas negras locales, detecta acortadores conocidos, identifica dominios con punycode o caracteres Unicode, y revisa parámetros de redirección sospechosos.

En tercer lugar, se estudia el contenido HTML. Se detectan formularios con acciones vacías, relativas o dirigidas a dominios sospechosos, así como elementos como iframe, base href, enlaces javascript: o redirecciones mediante meta refresh.

Por último, se analizan rasgos de ingeniería social presentes en el texto, como saludos genéricos, lenguaje urgente, asuntos sospechosos, solicitudes de credenciales o referencias a documentos adjuntos.

Clasificador neuronal

Además del sistema heurístico, se ha implementado un clasificador neuronal basado en scikit-learn. El modelo utiliza un pipeline formado por un vectorizador TF-IDF y un MLPClassifier. Los hiperparámetros están centralizados en HiperparametrosModelo y sus valores predeterminados se persisten en runtime/server/.env.local. La interfaz los consulta y modifica

mediante la API administrativa; un cliente puede enviar una excepción explícita al comparar o entrenar, pero no escribe el sistema de archivos del servidor.

El sistema permite una versión activa en español y otra en inglés. La vista de entrenamiento serializa los CSV y, solo cuando se solicita una excepción, los hiperparámetros; de lo contrario, el backend usa sus valores centrales persistidos. El servidor valida datos, entrena desde cero, guarda el artefacto de forma atómica en `runtime/server/models` e invalida la caché. Evaluación y comparación también se ejecutan en el servidor; los clientes reciben únicamente métricas y metadatos.

Los modelos entrenados se almacenan mediante `joblib` en `runtime/server/models`, diferenciando el artefacto español del inglés. Solo el backend accede a esas rutas. Durante el análisis, el servidor detecta el idioma con `langdetect` y reutiliza un detector cacheado por idioma. Si el modelo esperado no existe, está corrupto o tiene metadatos incompatibles, se entrena un modelo sintético del mismo idioma; nunca se reutiliza silenciosamente el modelo de la lengua opuesta.

Interfaz de usuario

El cliente web se desarrolla con `Streamlit` y organiza cinco vistas: Inicio, Configuración, Detección, Monitor y Entrenamiento. Comparte un sistema visual adaptable con cabecera de marca, navegación compacta, tarjetas de estado y componentes coherentes. Detección ordena fuente, configuración y resultado en tres pasos; Configuración agrupa conexiones, monitor, backend y red neuronal en pestañas. Sus responsabilidades son recoger entradas, llamar mediante `BackendClient` y presentar la respuesta, sin cargar modelos ni exponer rutas locales completas de credenciales.

El resultado se muestra mediante una puntuación de riesgo, una clasificación global y un panel de detalles. En el análisis heurístico, el usuario puede consultar qué señales se han activado y leer una explicación de cada una. Esta decisión busca mejorar la transparencia del sistema, evitando que el resultado sea una simple etiqueta opaca.

La integración con Gmail usa `OAuth` de solo lectura. La vista de detección obtiene EML autorizados y los remite al backend; el proceso `monitor_gmail.py` hace lo mismo periódicamente y puede notificar por Telegram. Ambos consumen la misma versión central y un fallo temporal queda controlado para reintento.

Diseño modular

Durante el desarrollo se ha realizado una refactorización orientada a `SOLID` y a una frontera cliente-servidor explícita. La presentación depende de `BackendClient`; el servidor separa transporte, casos de uso, señales, puntuación, explicaciones, datasets y aprendizaje. El almacenamiento sigue la misma frontera: `runtime/client` contiene secretos y preferencias propios de cada instalación, mientras `runtime/server` contiene configuración y modelos centrales. Así, los detalles del modelo permanecen fuera de `Streamlit`, extensión y monitor.

Esta organización permite que cada módulo tenga una responsabilidad principal. Por ejemplo, `SignalBuilder` no calcula puntuaciones, sino que únicamente construye señales; `RiskScorer` no conoce cómo se han obtenido las señales, solo calcula el riesgo; y `ExplanationBuilder` no decide si un correo es phishing, sino que transforma resultados técnicos en texto comprensible.

Tabla 5.2: Capas y responsabilidades del diseño modular.

Capa	Módulos principales	Responsabilidad
Presentación y clientes	app.py, detect_app.py, config_app.py, train_app.py, monitor_app.py y extension_gmail/	Recoger entradas, mantener el estado visual y presentar la respuesta del backend.
Transporte	backend_client.py y http_api.py	Serializar JSON, aplicar límites, gestionar CORS/autorización y devolver códigos HTTP.
Aplicación	backend_service.py y analysis_service.py	Coordinar análisis, modos, caché, ajustes, entrenamiento, evaluación y versiones.
Dominio de análisis	analizador_email.py, correo.py, signal_builder.py, scorer.py y explanations.py	Normalizar el correo, construir señales, calcular el riesgo y generar explicaciones.
Aprendizaje y datos	modelo_neural.py, model_config.py, dataset.py, training_protocol.py y metrics.py	Vectorizar, entrenar, inferir, limpiar datasets y calcular métricas reproducibles.
Infraestructura e integraciones	runtime_paths.py, env_loader.py, file_utils.py, gmail_client.py y telegram_notifier.py	Separar persistencia, escribir de forma atómica e integrar Gmail y Telegram.

Esta separación facilita la evolución del prototipo. Pueden añadirse reglas o sustituirse el clasificador en el servidor sin redistribuir los clientes. La versión revisada ejecuta correctamente 94 pruebas Python y 2 recorridos reales con Chromium; GitHub Actions repite pruebas, Ruff, calibración, evaluación reproducible, respaldo de defensa y navegación de interfaz.

Limitaciones

El prototipo presenta varias limitaciones. Las listas negras son locales y no se consultan servicios externos de reputación en tiempo real. Tampoco se realiza análisis dinámico de las páginas enlazadas, ni se validan certificados digitales ni se comprueba la reputación histórica de los dominios. Además, Streamlit se utiliza como interfaz académica en un entorno controlado: la versión actual no incorpora gestión multiusuario, control de acceso ni aislamiento de datos para un despliegue público.

El modelo neuronal depende de la calidad y representatividad de los datasets utilizados para el entrenamiento. Aunque la aplicación permite entrenar con CSV externos, su rendimiento real debe evaluarse con conjuntos amplios, recientes y equilibrados. Por último, el prototipo no incorpora transformers ni LLMs, aunque el estado del arte los identifica como una línea prometedora junto con la detección semántica de principios de ingeniería social.

Pruebas, resultados y discusión

Pruebas unitarias y de integración

La versión revisada ejecuta 94 pruebas unitarias y de integración mediante unittest. Cubren EML, señales BEC, combinación calibrada, selección determinista de idioma, cliente HTTP, entrenamiento central, Gmail, monitor, heurísticas, clasificador, persistencia separada, ajustes por API, autenticación administrativa, extensión, evaluación externa, respaldo de defensa y Telegram. Una prueba integral recorre Gmail EML, JSON, HTTP, backend, estado y alerta Telegram. Además, 2 pruebas con Chromium recorren la extensión y un análisis real entre Streamlit y backend. Todas finalizaron correctamente; los avisos de convergencia proceden de iteraciones reducidas del MLP en pruebas rápidas.

Rendimiento y eficiencia de arranque

Para cuantificar el coste operativo se añadió un benchmark reproducible que mide el arranque, el análisis heurístico, la detección de idioma, la carga de modelos y la inferencia. Cada microbenchmark se ejecutó con cinco repeticiones y se tomó la mediana por operación; las importaciones se midieron en procesos nuevos para evitar que la caché de módulos ocultase el coste de inicio.

El perfilado localizó el principal cuello de botella en las importaciones anticipadas. Importar únicamente las heurísticas requería 1098,5090 ms y, tras diferir la carga de scikit-learn y del subsistema neuronal, pasó a 37,0194 ms: una reducción del 96,6 % y una aceleración de 29,7 veces. El arranque frío de la aplicación Streamlit bajó de 1326,5726 ms a 315,1490 ms, una mejora del 76,2 % y 4,2 veces.

Las rutas de inferencia permanecieron estables: el análisis heurístico pasó de 0,1097 ms a 0,1084 ms, la detección de idioma de 2,8185 ms a 2,8230 ms, la predicción neuronal de 0,3101 ms a 0,3190 ms y el análisis combinado en caliente de 3,4187 ms a 3,3611 ms. Las variaciones de hasta el 3 % se consideran ruido de medición, ya que no se modificaron reglas, pesos ni modelos.

Diseño experimental y datasets

El protocolo reproducible fija las fuentes, licencias y SHA-256 sin versionar los mensajes. En español se combinan el split de entrenamiento de softcapps/spam_ham_spanish (Hugging Face, DOI 10.57967/hf/2264) y SMS Spam Mexico (Kaggle). Tras retirar 147 copias, dos filas contradictorias y una coincidencia con la prueba quedan 1.148 textos. Las 209 filas del split oficial se reservan para probar. En inglés se utiliza únicamente phishing_email.csv (Kaggle), porque ya agrega CEAS, Enron, Ling, Nazario, Nigerian Fraud y SpamAssassin: añadir también esos seis CSV duplicaba el corpus. Tras retirar 408 copias quedan 82.077 textos, divididos de forma estratificada 80/20 con semilla 42. Los joblib guardan el pipeline y estos metadatos, nunca el texto bruto.

Tabla 6.1: Conjuntos empleados y separación experimental.

Conjunto	N	Phishing	Legítimos	Procedencia	Uso
Entrenamiento ES	1.148	613	535	Hugging Face + SMS Spam Mexico	Ajuste del modelo español
Prueba ES	209	100	109	Split oficial softcapps	Holdout español

Conjunto	N	Phishing	Legítimos	Procedencia	Uso
Entrenamiento EN	65.661	34.275	31.386	80 % del agregado deduplicado	Ajuste del modelo inglés
Prueba EN	16.416	8.569	7.847	20 % estratificado del agregado	Holdout inglés
Calibración	40	20	20	Casos sintéticos ES/EN; 10 por idioma y clase	Cinco particiones para pesos y umbral
Evaluación EML	16	8	8	EML sintéticos reservados; 4 por idioma y clase	Comparación final de los tres modos
Validación Zenodo	100	23	77	100 textos únicos de 2.000 filas	Diagnóstico secundario
DIFrauD	1.528	608	920	Split público histórico de phishing	Diagnóstico externo con riesgo de solapamiento

La separación se realiza antes del ajuste. Español conserva el test oficial y elimina del entrenamiento cualquier coincidencia exacta; inglés deduplica primero y después divide 80/20 de forma estratificada y determinista. Otros 40 casos calibran la fusión mediante cinco particiones y 16 EML distintos comprueban los tres modos con MIME y cabeceras. Zenodo y DIFrauD añaden diagnósticos externos; DIFrauD no se considera independiente por posible solapamiento de fuentes históricas con el agregado inglés (Boumber et al., 2024).

evaluation/training_sources.json fija las URLs y huellas de los CSV externos. scripts/retrain_reproducible.py verifica esos archivos, reconstruye las mismas particiones, entrena y genera métricas y matrices de confusión. Los SMS españoles son una aproximación a smishing y la clase positiva inglesa también agrega spam; además, los holdouts de texto carecen de MIME. Estas limitaciones de validez de constructo impiden extrapolar las cifras a producción.

Resultados comparados

Tras fijar 45 % de peso heurístico, 55 % neuronal, umbral 21 y conservación de evidencia individual a partir de 70, los holdouts y los 16 EML se procesaron sin reajustar parámetros. Accuracy mide el acierto total, precisión la fiabilidad de las alertas, recall la cobertura del phishing real y F1 su equilibrio.

Tabla 6.2: Métricas sobre los holdouts de texto.

Idioma/modo	N	Accuracy	Precisión	Recall	F1
ES heurístico	209	52,2 %	0,0 %	0,0 %	0,0 %
ES neuronal	209	87,1 %	82,9 %	92,0 %	87,2 %
ES combinado	209	89,5 %	87,5 %	91,0 %	89,2 %
EN heurístico	16.416	48,0 %	92,7 %	0,4 %	0,9 %
EN neuronal	16.416	98,3 %	98,0 %	98,8 %	98,4 %
EN combinado	16.416	98,4 %	98,2 %	98,7 %	98,4 %

El heurístico apenas activa señales en texto plano porque no recibe cabeceras, HTML estructurado ni autenticación. Estas filas miden principalmente el modelo léxico y la fusión; la prueba EML posterior es la comparación funcional del sistema completo.

Tabla 6.3: Métricas sobre los 16 EML reservados.

Modo	Accuracy	Precisión	Recall	F1	Accuracy balanceada
Heurístico	100,0 %	100,0 %	100,0 %	100,0 %	100,0 %
Neuronal	81,2 %	77,8 %	87,5 %	82,3 %	81,2 %
Combinado 45/55	87,5 %	80,0 %	100,0 %	88,9 %	87,5 %

Tabla 6.4: Matrices de confusión sobre los 16 EML reservados.

Modo	VN	FP	FN	VP	Lectura
Heurístico	8	0	0	8	Clasifica correctamente los 16 escenarios
Neuronal	6	2	1	7	Dos falsas alarmas y un phishing omitido
Combinado 45/55	6	2	0	8	Detecta los 8 phishing y genera dos falsas alarmas

El heurístico obtiene el mejor resultado en este conjunto pequeño porque los EML contienen las cabeceras, enlaces y escenarios para los que se diseñaron las reglas. El combinado conserva los ocho verdaderos positivos y reduce el riesgo de depender solo del texto, a costa de dos falsos positivos. El neuronal genera dos falsas alarmas y omite un phishing. En los diagnósticos externos, el combinado alcanza 70,0 % de accuracy y 95,7 % de recall sobre los 100 textos únicos de Zenodo, y 89,0 % de accuracy y 93,4 % de recall sobre DIFrauD. La diferencia frente al 98,4 % del holdout inglés revela cambio de distribución y refuerza que no existe un ganador universal ni una métrica extrapolable a producción.

Comportamiento del sistema heurístico

La mezcla ponderada original podía diluir una evidencia fuerte. Con los modelos reproducibles reentrenados, la calibración separada asignó 45 % a heurística y 55 % al modelo, fijó el umbral 21 y mantuvo alta confianza 70. Los indicios BEC aislados pesan poco y solo la coincidencia de los tres recibe el refuerzo de alta confianza. Los dos BEC sin enlace reservados se detectan. Esto resuelve esos escenarios concretos, no todo BEC posible.

Prueba funcional de la aplicación web

Se realizaron 94 pruebas Python y 2 recorridos Chromium. Las comprobaciones verifican los tres modos, calibración, idioma determinista, EML serializable, Gmail a backend y Telegram, cliente HTTP, entrenamiento y versión central, separación de datos, ajustes centrales, caché, señales BEC, API, límites, token administrativo, evaluación externa, respaldo, extensión y Streamlit. Los 16 EML y DIFrauD añaden medición separada con sus límites expresos; no se presentan como evaluación estadística de producción.

Evidencia visual del prototipo

Las siguientes capturas se generaron de forma reproducible con el backend y Streamlit en procesos separados y un navegador Chromium real. El recorrido muestra las entradas admitidas, los modos combinado y heurístico, las explicaciones, las versiones activas ES/EN y los ajustes compartidos del servidor. Se utilizó exclusivamente el EML BEC sintético reservado, sin credenciales, cuentas ni correos personales.



Figura 6.1: Pantalla inicial del cliente web con el backend central conectado. Fuente: elaboración propia.

Backend conectado <http://127.0.0.1:8766>

ES - activo

EN - activo

01

Selecciona la fuente
Puedes pegar el mensaje, subir su archivo original o importarlo desde Gmail.

Modo de entrada

☒ Pegar texto del correo ☐ Subir archivo .eml ☐ Analizar correos de Gmail

Pega aquí el contenido del correo (cabeceras + cuerpo):

From: remitente@dominio.com
Subject: Asunto del mensaje

Pega aquí el cuerpo completo del correo...

02

Configura el análisis
El modo combinado ofrece el equilibrio recomendado entre explicación y modelo.

Tipo de análisis

☐ Heurístico ☐ Red neuronal ☒ Combinado

Peso heurístico (%)

45

Fusión aplicada: 45% heurística - 55% neuronal

03

Ejecuta y revisa el resultado
El backend devolverá el veredicto, la puntuación y la evidencia disponible.

Analizar correo

Figura 6.2: Fuentes de entrada y configuración del modo combinado en el cliente web. Fuente: elaboración propia.

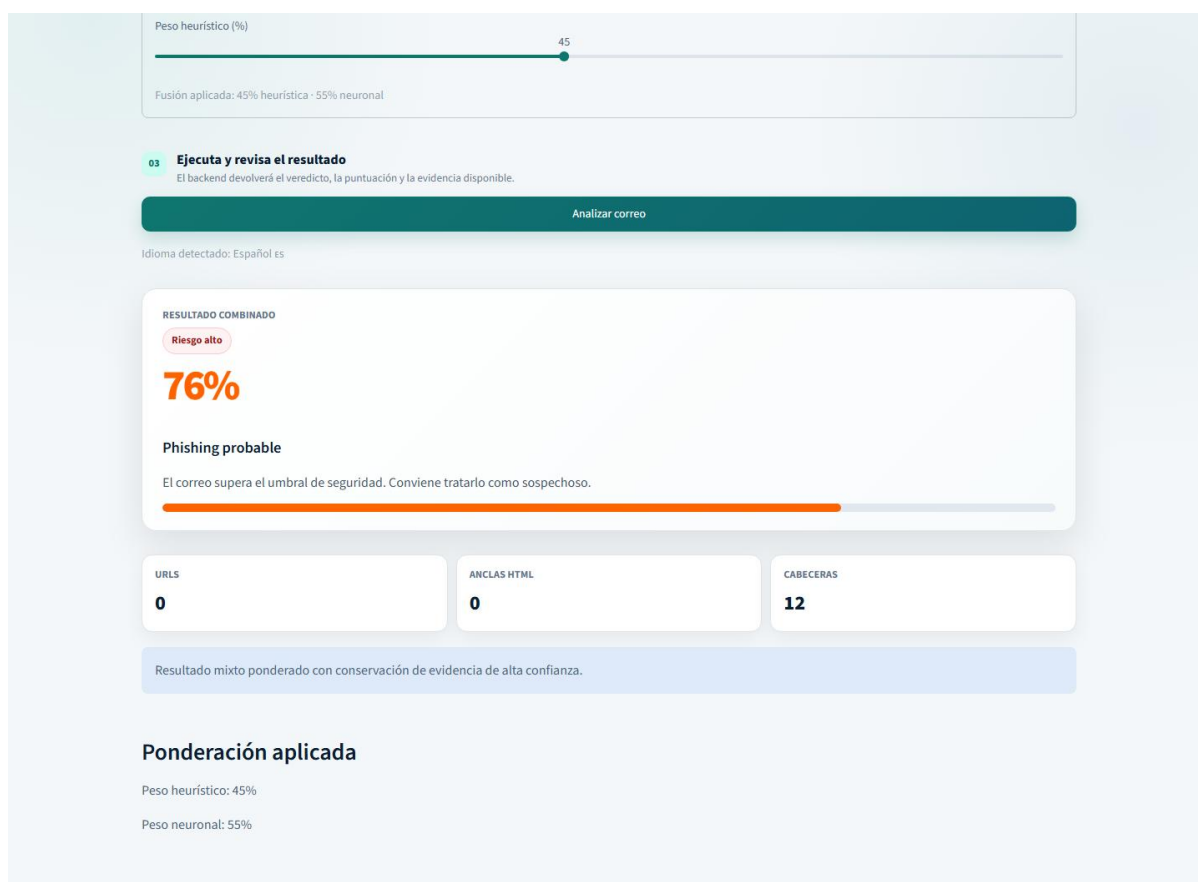


Figura 6.3: Resultado combinado del escenario BEC sintético reservado. Fuente: elaboración propia.

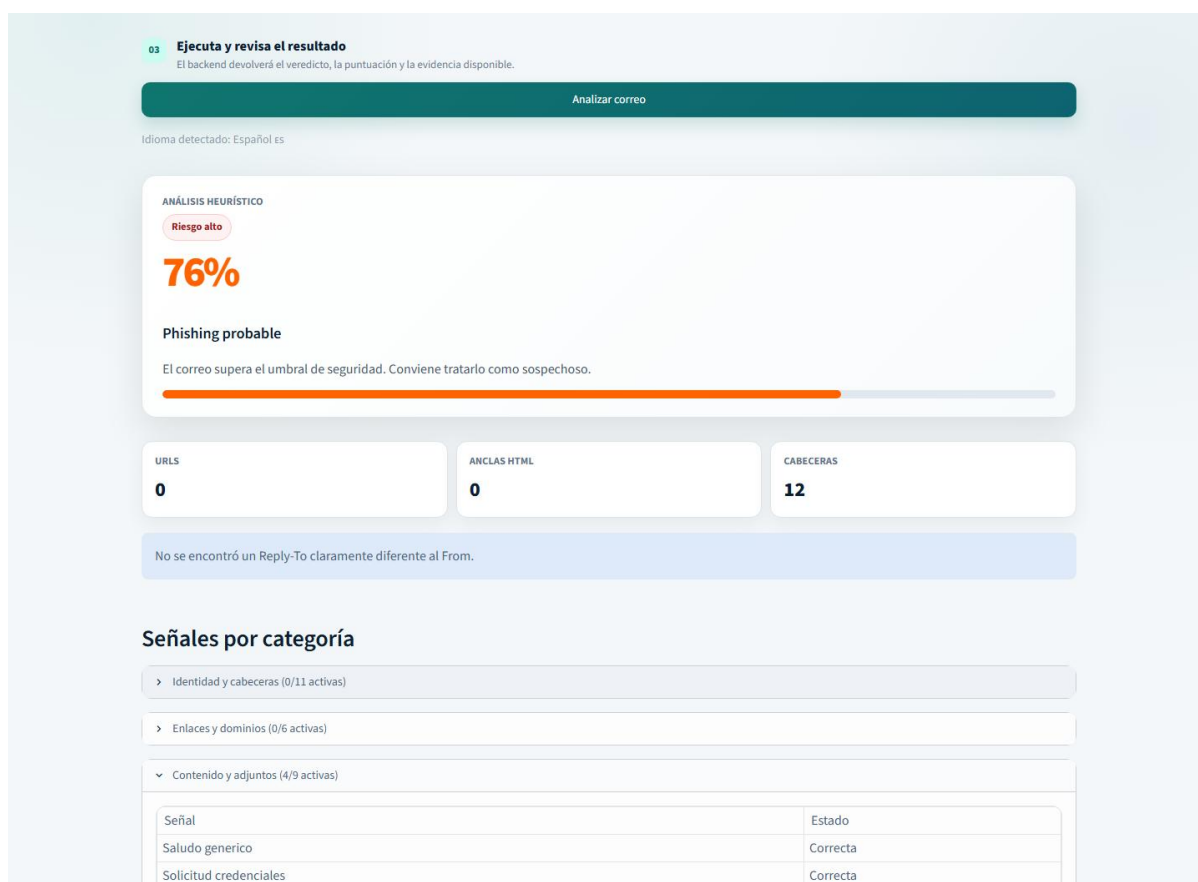


Figura 6.4: Resultado heurístico y desglose explicable de señales por categoría. Fuente: elaboración propia.

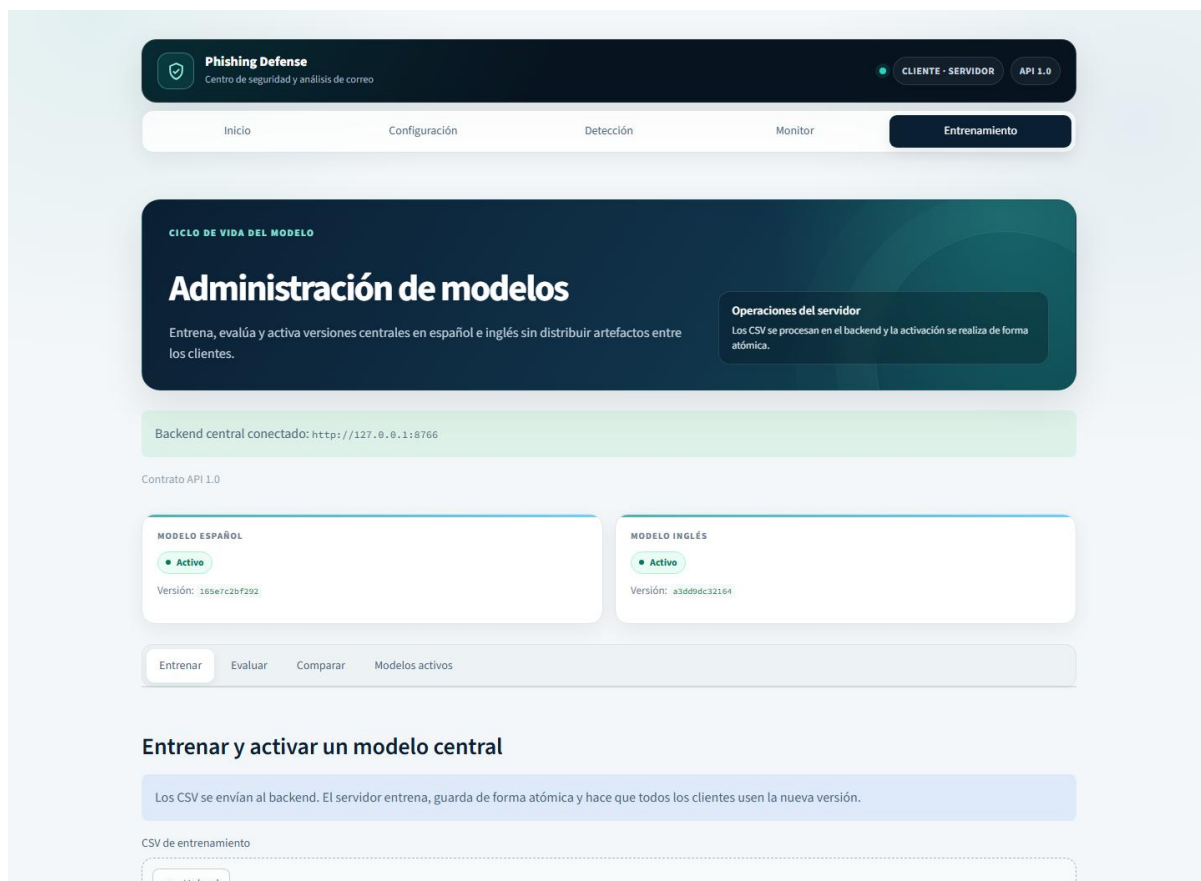


Figura 6.5: Administración y versiones activas de los modelos centrales ES y EN. Fuente: elaboración propia.

modalidades según personalización y técnicas emergentes contrastadas con fuentes académicas e informes de industria.

El segundo objetivo, estudiar los fundamentos psicológicos de la ingeniería social, se cubre mediante el análisis de los principios de Cialdini, los sesgos cognitivos de Kahneman y Tversky y el impacto de la inteligencia artificial generativa.

El tercer objetivo, revisar el estado del arte en técnicas de detección, se desarrolla en la sección dedicada a machine learning clásico, deep learning, transformers y enfoques híbridos, y concluye con el hueco de investigación que justifica el prototipo.

El cuarto objetivo, diseñar e implementar un sistema heurístico explicable y un clasificador TF-IDF + MLP, se materializa en el motor de señales sobre cabeceras, contenido, URLs y HTML y en el pipeline neuronal entrenable en español e inglés.

El quinto objetivo, integrar la aplicación con Gmail y proporcionar alertas mediante Telegram, se cumple mediante OAuth de solo lectura y el monitor opcional. Los mensajes autorizados se envían al backend central y Telegram recibe únicamente la alerta configurada; ni Streamlit ni el monitor duplican la lógica de análisis.

El sexto objetivo, evaluar el comportamiento del sistema, se aborda con 94 pruebas Python, 2 recorridos Chromium —incluido web a backend—, benchmark reproducible, calibración separada de 40 casos, 16 EML reservados y 1.528 textos externos con hashes, matriz de confusión y errores por identificador. La estimación independiente sobre correo real reciente sigue delimitada como trabajo pendiente.

Entre los logros principales destacan la arquitectura cliente-servidor, una versión activa por idioma compartida por todos los clientes, activación atómica, Gmail/Telegram, pruebas Python y de navegador, CI, reentrenamiento reproducible, calibración separada, detección BEC y evaluación trazable local/externa. La carga diferida reduce históricamente el arranque de heurísticas un 96,6 % y el de la aplicación un 76,2 %. La fusión 45/55 conserva evidencia de alta confianza.

Trabajo futuro

Como trabajo futuro prioritario se plantea confirmar la calibración sobre datasets reales, recientes e independientes en español e inglés, con licencia, procedencia temporal y deduplicación frente al entrenamiento, y ampliar la detección BEC más allá de los patrones actuales. También se estudiarán ataques adversariales, transformers/LLM, reputación de dominios, análisis dinámico y etiquetado seguro. Un despliegue multiusuario requerirá autenticación, TLS, rate limiting, secretos e aislamiento de sesiones.

Glosario

API: Interfaz de operaciones que permite a un cliente enviar datos al backend mediante un contrato HTTP; Gmail y Telegram son integraciones externas distintas.

Aplicación web cliente-servidor: El navegador usa Streamlit como capa de presentación y Streamlit llama por HTTP al backend central obligatorio. En la configuración académica ambos procesos se ejecutan en el mismo equipo; la extensión y el monitor consumen el mismo contrato.

Blacklist (lista negra): Lista local de dominios o URLs considerados no confiables que el sistema consulta durante el análisis.

Business Email Compromise (BEC): Fraude que suplanta a directivos o proveedores para autorizar pagos o transferencias fraudulentas.

DKIM: Mecanismo de autenticación de correo basado en firmas criptográficas del dominio remitente.

DMARC: Política que indica a los servidores receptores cómo tratar los mensajes que no superan SPF o DKIM.

Análisis combinado: Modo que pondera la puntuación heurística y la probabilidad del clasificador neuronal.

Streamlit: Framework de Python utilizado para construir la interfaz web interactiva del prototipo.

Heurística: Conjunto de reglas basadas en conocimiento experto que evalúan indicios de phishing de forma explicable.

MLPClassifier: Perceptrón multicapa de scikit-learn: red neuronal de alimentación directa con una o más capas ocultas.

OAuth: Protocolo de autorización empleado para acceder a la API de Gmail sin compartir la contraseña.

Phishing: Técnica de ingeniería social que suplanta entidades legítimas para obtener información sensible.

Precisión: Proporción de casos clasificados como phishing que realmente lo son.

Quishing: Variante de phishing que utiliza códigos QR como vector de entrega.

Recall: Proporción de los casos de phishing reales que el modelo detecta correctamente.

Smishing: Variante de phishing mediante mensajes SMS.

TF-IDF: Técnica de vectorización de texto que pondera la frecuencia de cada término según su importancia.

Umbral (threshold): Valor de puntuación a partir del cual un correo se considera phishing probable.

Vishing: Variante de phishing mediante llamadas telefónicas.

Referencias

- AlEroud, A., & Karabatis, G. (2020). Bypassing detection of URL-based phishing attacks using generative adversarial deep neural networks. In *Proceedings of the Sixth International Workshop on Security and Privacy Analytics (IWSPA '20)* (pp. 53-60). ACM. <https://doi.org/10.1145/3375708.3380315>
- Alghenaim, M., Alkaws, G., & Barnhart, C. R. (2025). The state of the art in AI-based phishing detection: A systematic literature review. In M. A. Al-Sharafi, M. Al-Emran, M. A. Mahmoud, & I. Arpacı (Eds.), *Current and Future Trends on AI Applications* (Studies in Computational Intelligence, Vol. 1178, pp. 431-458). Springer. https://doi.org/10.1007/978-3-031-75091-5_23
- Alkhalil, Z., Hewage, C., Nawaf, L., & Khan, I. (2021). Phishing attacks: A recent comprehensive study and a new anatomy. *Frontiers in Computer Science*, 3, Article 563060. <https://doi.org/10.3389/fcomp.2021.563060>
- Altawairy, N., Al-Turaiki, I., Alotaibi, R., & Alakeel, F. (2024). Advancing phishing email detection: A comparative study of deep learning models. *Sensors*, 24(7), 2077. <https://doi.org/10.3390/s24072077>
- Anti-Phishing Working Group. (2025). Phishing Activity Trends Report – 2nd Quarter 2025. https://docs.apwg.org/reports/apwg_trends_report_q2_2025.pdf
- Boumber, D. A., Qachfar, F. Z., & Verma, R. (2024). Domain-agnostic adapter architecture for deception detection: Extensive evaluations with the DIFraudD benchmark. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)* (pp. 5260-5274). ELRA and ICCL. <https://aclanthology.org/2024.lrec-main.468/>
- Cialdini, R. B. (1984). *Influence: The Psychology of Persuasion*. William Morrow.
- Federal Bureau of Investigation. (2024). 2023 Internet Crime Report. Internet Crime Complaint Center. https://www.ic3.gov/Media/PDF/AnnualReport/2023_IC3Report.pdf
- Fortinet. (2025). 19 types of phishing attacks with examples. <https://www.fortinet.com/resources/cyberglossary/types-of-phishing-attacks>
- GreatHorn. (2021). 3 phases of the phishing attack kill chain. <https://web.archive.org/web/20231201201455/https://www.greathorn.com/blog/what-are-the-3-phases-of-the-phishing-attack-kill-chain/>
- Hadnagy, C. (2018). *Social Engineering: The Science of Human Hacking* (2nd ed.). Wiley.
- Heiding, F., Schneier, B., Vishwanath, A., Bernstein, J., & Park, P. S. (2024). Devising and detecting phishing emails using large language models. *IEEE Access*, 12, 42131-42146. <https://doi.org/10.1109/ACCESS.2024.3375882>
- Hoxhunt. (2024). The Phishing Kill Chain: Three Steps and Five Effects. <https://hoxhunt.com/blog/three-steps-and-five-effects-of-the-phishing-attack-kill-chain>
- Hutchinson, S., Zhang, Z., & Liu, Q. (2018). Detecting phishing websites with random forest. In *Machine Learning and Intelligent Communications* (pp. 470-479). Springer. https://doi.org/10.1007/978-3-030-00557-3_46
- IBM. (2025). Cost of a Data Breach Report 2025. <https://www.ibm.com/reports/data-breach>
- Imperva. (2023). What is phishing: Attack techniques & scam examples. <https://www.imperva.com/learn/application-security/phishing-attack-scam/>
- Instituto Nacional de Ciberseguridad. (2025). Balance de ciberseguridad 2024. https://www.incibe.es/sites/default/files/Comunicación_2025/Infografía_BalanceCiberseguridad_INCIBE_2024_web.pdf
- Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.
- Koide, T., Fukushi, N., Nakano, H., & Chiba, D. (2024). ChatSpamDetector: Leveraging large language models for effective phishing email detection. arXiv preprint arXiv:2402.18093. <https://arxiv.org/abs/2402.18093>
- Kroll. (2025). Q4 2024 Cyber Threat Landscape Report: CorruptQR Campaign. <https://www.kroll.com/en/reports/cyber/threat-intelligence-reports/q4-2024-threat-landscape-report-phishing>
- Kuikel, S., Piplai, A., & Aggarwal, P. (2025). Evaluating large language models for phishing detection, self-consistency, faithfulness, and explainability. arXiv preprint arXiv:2506.13746. <https://arxiv.org/abs/2506.13746>
- Le, H., Pham, Q., Sahoo, D., & Hoi, S. C. H. (2018). URLNet: Learning a URL representation with deep learning for malicious URL detection. arXiv preprint arXiv:1802.03162. <https://arxiv.org/abs/1802.03162>
- Microsoft. (2025). Microsoft Digital Defense Report 2025. <https://www.microsoft.com/en-us/corporate-responsibility/cybersecurity/microsoft-digital-defense-report-2025/>

- MITRE. (2025). ATT&CK Framework: T1566 Phishing. <https://attack.mitre.org/techniques/T1566/>
- Popescu, D., & Radu, L. D. (2025). AI in phishing detection: A bibliometric review. *Frontiers in Artificial Intelligence*, 8, 1496580. <https://doi.org/10.3389/frai.2025.1496580>
- Proofpoint. (2025). The Human Factor 2025: Vol. 1 Social Engineering. <https://www.proofpoint.com/us/resources/threat-reports/human-factor-social-engineering>
- Rojas-Galeano, S. (2024). Zero-shot spam email classification using pre-trained large language models. *arXiv preprint arXiv:2405.15936*. <https://arxiv.org/abs/2405.15936>
- Sánchez-Paniagua, M., Fidalgo, E., Alegre, E., & Alaiz-Rodríguez, R. (2022). Phishing websites detection using a novel multipurpose dataset and web technologies features. *Expert Systems with Applications*, 207, 118010. <https://doi.org/10.1016/j.eswa.2022.118010>
- Thakur, K., Ali, M. L., Obaidat, M. A., & Kamruzzaman, A. (2023). A systematic review on deep-learning-based phishing email detection. *Electronics*, 12(21), 4545. <https://www.mdpi.com/2079-9292/12/21/4545>
- Trad, F., & Chehab, A. (2024). Prompt engineering or fine-tuning? A case study on phishing detection with large language models. *Machine Learning and Knowledge Extraction*, 6(1), 367-384. <https://doi.org/10.3390/make6010018>
- Tversky, A., & Kahneman, D. (1974). Judgment under uncertainty: Heuristics and biases. *Science*, 185(4157), 1124-1131. <https://doi.org/10.1126/science.185.4157.1124>
- Uddin, M. A., Mahiuddin, M., & Sarker, I. H. (2024). An explainable transformer-based model for phishing email detection: A large language model approach. *arXiv preprint arXiv:2402.13871*. <https://arxiv.org/abs/2402.13871>
- Verizon. (2025). 2025 Data Breach Investigations Report. <https://www.verizon.com/business/resources/reports/2025-dbir-executive-summary.pdf>
- Wang, Y., Zhu, W., Xu, H., Qin, Z., Ren, K., & Ma, W. (2023). A large-scale pretrained deep model for phishing URL detection. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 1-5). IEEE. <https://doi.org/10.1109/ICASSP49357.2023.10095719>
- Yerima, S. Y., & Alzaylaee, M. K. (2020). High accuracy phishing detection based on convolutional neural networks. In *Proceedings of the 3rd International Conference on Computer Applications & Information Security* (pp. 1-6). IEEE. <https://doi.org/10.1109/ICCAIS48893.2020.9096869>

Anexos

Este capítulo reúne el material complementario del Trabajo Fin de Grado: la estructura completa del repositorio, los fragmentos de código fuente más representativos del sistema y una guía resumida de instalación y uso. El código completo del prototipo se encuentra disponible en el repositorio GitHub asociado a este trabajo.

Anexo A: Estructura del repositorio

La siguiente estructura recoge los componentes principales de la aplicación cliente-servidor, sus clientes y el backend central de análisis:

Estructura de directorios del repositorio

```
TFG/
├── src/
│   ├── app.py                # Entrada principal de Streamlit
│   ├── backend_server.py     # Servidor HTTP central
│   ├── config_app.py         # Configuración de clientes y servidor
│   ├── detect_app.py         # Cliente web de detección
│   ├── gmail_extension_server.py # Proxy legado opcional (puerto 8765)
│   ├── monitor_app.py        # Vista de monitorización de Gmail
│   ├── monitor_gmail.py      # Monitorización periódica opcional
│   ├── train_app.py          # Entrenamiento y evaluación
│   ├── ui_components.py      # Componentes visuales compartidos
│   └── sistema_phishing/
│       ├── analizador_email.py # Parseo y normalización de correos
│       ├── analysis_service.py  # Coordinación del análisis en servidor
│       ├── analyzer.py          # Orquestador heurístico
│       ├── backend_service.py   # Casos de uso del servidor central
│       ├── backend_client.py    # Cliente HTTP compartido
│       ├── model_config.py      # Configuración ML ligera para clientes
│       ├── file_utils.py        # Escrituras atómicas de secretos y estado
│       └── network.py           # Política de loopback y bind
```

— configuracion.py	# Configuración del dominio
— content_signals.py	# Señales de texto y adjuntos
— correo.py	# Representación interna común
— dataset.py	# Normalización de CSV
— env_loader.py	# Variables locales de entorno
— explanations.py	# Explicaciones del riesgo
— gmail_client.py	# Integración OAuth con Gmail
— gmail_monitor.py	# Lógica de monitorización
— header_signals.py	# Cabeceras y autenticación
— heurísticas.py	# Fachada heurística
— html_signals.py	# Señales HTML
— http_api.py	# Transporte HTTP y validación
— idioma.py	# Detección de idioma
— metrics.py	# Métricas y matriz de confusión
— modelo_neural.py	# Pipeline TF-IDF + MLP
— neural.py	# Fachada neuronal
— scorer.py	# Cálculo de puntuación
— signal_builder.py	# Construcción de señales
— signals.py	# Utilidades de señales
— telegram_notifier.py	# Alertas por Telegram
— url_utils.py	# Análisis de URLs
— tests/	# Suite unittest
— scripts/	# Automatización y experimentos
— requirements.txt	
— credentials.example.json	
— .env.example	# Aviso de migración
— config/	# Plantillas cliente/servidor
— runtime/	# Datos persistentes separados
— client/	# Secretos y preferencias (Git ignora)
— server/	# Ajustes y modelos centrales
— models/	# Artefactos ES/EN

Anexo B: Fragmentos de código fuente

A continuación se reproducen los fragmentos más representativos del sistema: la fachada del análisis heurístico, el servicio común que combina heurística y red neuronal, y el núcleo del pipeline neuronal TF-IDF + MLP.

B.1. Fachada del análisis heurístico (heurísticas.py)

```
heurísticas.py
"""Fachada de heurísticas para el sistema de detección de phishing."""
from .analyzer import PhishingAnalyzer
from .correo import CorreoAnalizado
from .url_utils import extraer_urls
__all__ = ["analizar_correo", "extraer_urls"]
def analizar_correo(correo):
    """Normaliza la entrada y delega el análisis en PhishingAnalyzer."""
    correo_analizado = CorreoAnalizado.from_input(correo)
    return PhishingAnalyzer(correo_analizado).analyze()
```

B.2. Servicio común de análisis (analysis_service.py)

```
analysis_service.py (extracto actualizado)
from collections.abc import Callable
from threading import Lock
from typing import Protocol
from .analizador_email import construir_texto_para_analisis
from .heurísticas import analizar_correo
from .idioma import detectar_idioma_correo
from .modelo_neural import ModelStorage, NeuralPhishingClassifier, NeuralPhishingDetector
MODO_HEURISTICO = "heurístico"
MODO_NEURAL = "neural"
MODO_COMBINADO = "combinado"
def cargar_detector_neural(config, idioma="es"):
    ruta_principal = config.model_path_en if idioma == "en" else config.model_path_es
    classifier = ModelStorage(ruta_principal).load()
    idioma_esperado = "english" if idioma == "en" else "spanish"
    if classifier is not None and getattr(classifier, "language", None) != idioma_esperado:
        classifier = None # No reutilizar silenciosamente otro idioma.
    if classifier is None:
```



```

        classifier = NeuralPhishingClassifier(language=idioma_esperado)
        classifier.fit_default()
    return NeuralPhishingDetector(classifier)
class EmailAnalysisService:
    def __init__(self, config, heuristic_analyzer=None, detector_loader=None,
language_detector=None):
        validar_configuracion(config)
        self.config = config
        self._heuristic_analyzer = heuristic_analyzer or analizar_correo
        self._detector_loader = detector_loader or cargar_detector_neural
        self._language_detector = language_detector or detectar_idioma_correo
        self._detectores = {}
        self._detector_lock = Lock()
    # _aplicar_umbral y construir_resultado_combinado validan el contrato y los pesos.
    def analyze(self, datos_email):
        if self.config.mode == MODO_HEURISTICO:
            return _aplicar_umbral(self._heuristic_analyzer(datos_email), self.config.threshold)
        resultado_neural = self._analyze_neural(datos_email)
        if self.config.mode == MODO_NEURAL:
            return _aplicar_umbral(resultado_neural, self.config.threshold)
        resultado_heur = self._heuristic_analyzer(datos_email)
        return construir_resultado_combinado(resultado_heur, resultado_neural, self.config)
    def analyze_all(self, datos_email):
        resultado_heur = _aplicar_umbral(self._heuristic_analyzer(datos_email),
self.config.threshold)
        resultado_neural = _aplicar_umbral(self._analyze_neural(datos_email),
self.config.threshold)
        resultado_combinado = construir_resultado_combinado(resultado_heur, resultado_neural,
self.config)
        return {"heuristico": resultado_heur, "neural": resultado_neural, "combinado":
resultado_combinado}
    def _analyze_neural(self, datos_email):
        texto = construir_texto_para_analisis(datos_email)
        idioma = self._language_detector(texto)
        detector = self._detectores.get(idioma)
        if detector is None:
            with self._detector_lock:
                detector = self._detectores.get(idioma)
                if detector is None:
                    detector = self._detector_loader(self.config, idioma)
                    self._detectores[idioma] = detector
        return detector.analyze(texto, datos_email.get("from", ""), datos_email.get("subject",
""))

```

B.3. Núcleo del clasificador neuronal (modelo_neural.py)

modelo_neural.py (extracto actualizado)

```

class NeuralPhishingClassifier:
    def __init__(self, language="spanish", hiperparametros=None):
        self.language = language
        hp = hiperparametros or cargar_hiperparametros_desde_env()
        self.pipeline = Pipeline([
            ("vectorizer", TfidfVectorizer(ngram_range=hp.tfidf_ngram_range, ...)),
            ("classifier", MLPClassifier(hidden_layer_sizes=hp.mlp_hidden_layer_sizes, ...)),
        ])
    def fit(self, textos, etiquetas):
        # Valida las etiquetas y entrena una ejecución nueva.
        self.pipeline.fit(textos, etiquetas)
    def predict_proba(self, texts):
        proba = self.pipeline.predict_proba(texts)
        clases = self.pipeline.named_steps["classifier"].classes_.tolist()
        indice_phishing = clases.index(1)
        return [float(fila[indice_phishing]) for fila in proba]
    def __getstate__(self):
        state = self.__dict__.copy()
        state["training_texts"] = []
        state["training_labels"] = []
        return state
class NeuralPhishingDetector:
    def __init__(self, classifier):
        self.classifier = classifier
    def analyze(self, texto, remitente="", subject=""):
        probability = self.classifier.predict_proba([texto])[0]
        return {

```

```

        "is_phishing": probability >= 0.5,
        "risk_score": round(probability * 100, 1),
        "description": "Clasificación basada en una red neuronal entrenada.",
        "from": remitente,
        "subject": subject,
    }

```

Anexo C: Guía resumida de instalación y uso

Este anexo resume la ejecución cliente-servidor local. Primero se inicia el backend obligatorio en 127.0.0.1:8766 y después Streamlit en 127.0.0.1:8501. Para una demostración desde un móvil de la misma red privada se mantiene la API en loopback y se inicia solo la web con `streamlit run src/app.py --server.address 0.0.0.0 --server.port 8501`; se accede mediante `http://IP_DEL_EQUIPO:8501` y puede ser necesario permitir el puerto 8501 en el perfil privado del firewall. No requiere `--allow-remote` ni abrir 8766. Como Streamlit no incorpora autenticación multiusuario, este acceso LAN debe ser temporal, en una red de confianza y sin redirección de puertos. La extensión apunta directamente al backend desde Opciones y el monitor usa `PHISHING_BACKEND_URL`. El puerto 8765 queda como proxy antiguo opcional. Un despliegue remoto real exige HTTPS y controles operativos adicionales.

Comandos de instalación y uso

```

# 1. Crear y activar un entorno virtual
python -m venv .venv
.\venv\Scripts\Activate.ps1

# 2. Instalar dependencias
python -m pip install -r requirements-dev.txt -c constraints.txt

# 3. Ejecutar primero el backend central
$env:PYTHONPATH="src"; python src/backend_server.py

# 3b. Ejecutar el cliente web en otra terminal
$env:PYTHONPATH="src"; streamlit run src/app.py
# 3c. El proxy 8765 solo se usa por compatibilidad antigua
$env:PYTHONPATH="src"; python src/gmail_extension_server.py
# 4. Ejecutar la suite de pruebas
$env:PYTHONPATH="src"; python -m unittest discover -s tests -p "test_*.py"

```

C.1. Uso del prototipo

La aplicación ofrece texto, EML y Gmail desde Streamlit, además del correo visible en la extensión. Todos esos clientes envían la entrada al backend central. `/analyze` acepta campos JSON o EML Base64 y devuelve riesgo, señales, explicación, idioma y versión para que cada cliente se limite a presentarlos.

Para cada correo, el sistema devuelve una puntuación global de riesgo, una clasificación y un desglose de señales. Interpreta los resultados SPF/DKIM/DMARC ya presentes en las cabeceras, sin consultas DNS ni validación criptográfica, y revisa coherencia del remitente, urgencia, URLs, IPs, punycode, acortadores, redirecciones, adjuntos e ingeniería social. Las señales activadas se acompañan de explicaciones para interpretar por qué se ha clasificado el mensaje.

El clasificador neuronal del backend asigna la probabilidad y selecciona el modelo por idioma. Streamlit recibe conjuntamente la puntuación neuronal y las señales heurísticas; no carga el pipeline ni recalcula la decisión.

Desde la vista Monitor se consultan correos de Gmail y se envían al backend. Si Telegram está configurado, el cliente puede alertar con el remitente, asunto y puntuación devueltos por el servidor.

Un caso habitual consiste en levantar backend y web, cargar un EML o pegar texto. El cliente envía la entrada, el servidor la procesa con la versión central y devuelve señales y riesgo. Para separar físicamente ambos lados se publica el backend detrás de HTTPS y se cambia PHISHING_BACKEND_URL; la latencia dependerá entonces también de la red.

Anexo D: Lista de verificación de la validación

La siguiente lista resume las comprobaciones realizadas sobre el prototipo durante la fase de validación, tal y como se detalla en el capítulo de pruebas y resultados.

Verificado: 94 pruebas Python y 2 recorridos reales con Chromium, incluido cliente web a backend.

Verificado: Benchmark reproducible de arranque e inferencia con comparación antes/después.

Verificado: Normalización correcta de etiquetas phishing y correo legítimo.

Verificado: Entrenamiento del modelo TF-IDF + MLP en español e inglés.

Verificado: calibración controlada de 40 casos y evaluación final de 16 EML locales reservados, con hashes, métricas y errores por caso; pendiente corpus real representativo.

Verificado: Cálculo de exactitud, precisión, recall y F1.

Verificado: arranque separado de backend y Streamlit, navegación y análisis HTTP completo.

Verificado: Análisis desde texto pegado y archivo .eml.

Verificado: Importación autorizada de correos mediante la API de Gmail.

Verificado: Selección del modelo neuronal según el idioma del mensaje.

Verificado: Monitorización de Gmail y notificación opcional por Telegram.

Anexo E: Resumen de arquitectura y evidencia

El sistema es cliente-servidor. Streamlit, la extensión y el monitor son clientes HTTP; backend_server.py centraliza parser, análisis, entrenamiento y una versión activa por idioma. En la configuración de defensa todos los procesos están en el mismo equipo y loopback, pero el servidor es obligatorio y puede separarse cambiando PHISHING_BACKEND_URL.

La evidencia reproducible comprende 94 pruebas Python, 2 recorridos con Chromium, integración continua, benchmark, calibración separada de 40 casos, evaluación de 16 EML reservados y diagnóstico de 1.528 textos DIFrauD. Los EML son sintéticos y DIFrauD conserva riesgo de solapamiento; ninguna cifra estima producción.

La persistencia se separa por propietario: runtime/client guarda las credenciales OAuth, el token, el estado del monitor y las preferencias de cada instalación; runtime/server guarda los ajustes centrales y los modelos ES/EN. GET/POST /settings permite administrarlos sin compartir el sistema de archivos del servidor.